

Chapter 3.1

User Interfaces

Contents

- Basic Widgets
- Stateless and Stateful Widget
- Design Systems
 - Cupertino Widgets
 - Material Widgets
- Layout
 - Single-child
 - Multi-child
- Sliver Widgets
- Performance best practices

Basic Widgets

Widgets

Widgets are UI components of your app. They define the :

- **A**ppearance
- **L**ayout, and
- **B**ehavior of a specific part of your app's interface

A widget represents a **single** element (e.g. button, text field, image) or even a **complex layout** container.

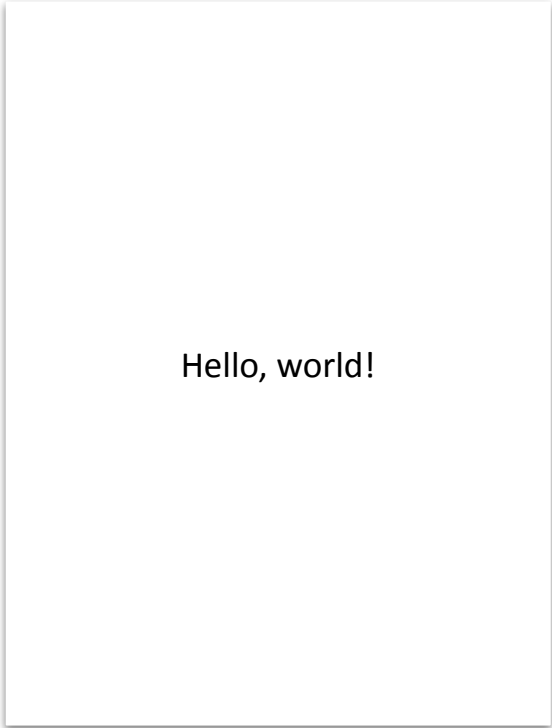
Widgets are **nested hierarchically**, forming a tree-like structure that defines the overall UI of your app.

Widgets

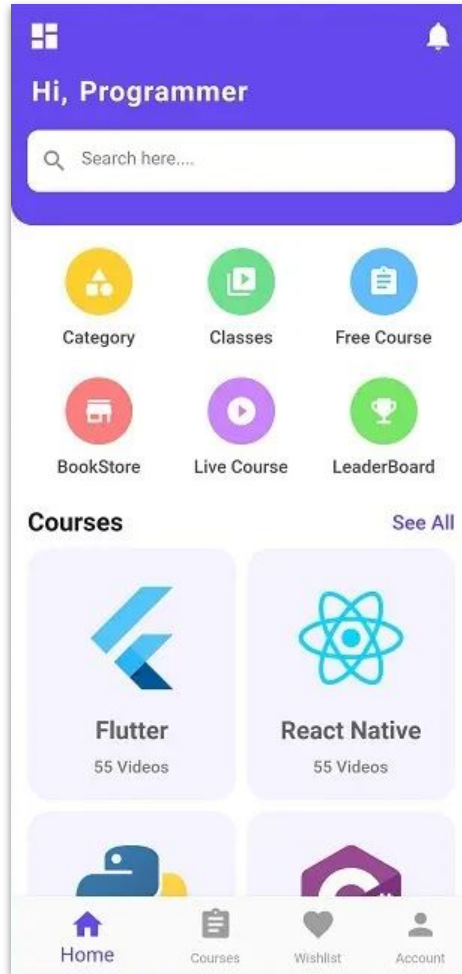
The minimum Flutter app with a text widget:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const Center(
      child: Text(
        'Hello, world!',
        textDirection: TextDirection.ltr,
      ),
    ),
  );
}
```



Hello, world!



Column

- Container
 - Column
 - Text
 - Search Bar
- Column
 - Row
 - Row
- Text
- Wrap
 - ListView
 - Card

Widget - Characteristics

Customizable:

- Most widgets offer various properties that you can adjust to customize their appearance and behavior.
- This allows you to create unique and personalized UIs.

Widget - Characteristics

Reusable:

- Widgets are designed to be reused throughout your app, promoting code efficiency and consistency.

```
ElevatedButton( onPressed: () {  
    // Handle button press },  
    child: Text('Button 1'), ),  
    // ... other parts of your app
```

```
CustomButton( text: 'Custom Button',  
    onPressed: () { // Handle button press },  
    color: Colors.green, ),
```


Basic widgets

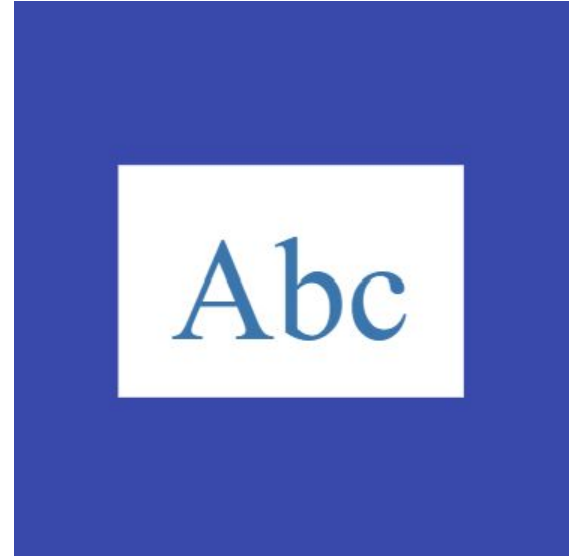
1. Text
2. TextField
3. ElevatedButton
4. Image
5. Row/Column
6. Scaffold
7. Placeholder
8. Icon
9. AppBar

Text

Text

Displays text content.

A run of text with a single style.



Text

The Text widget displays a string of text with single style.

The style argument is optional.

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    Text(
      'Hello, world!',
      textDirection: TextDirection.ltr,
    ),
  );
}
```

Text

```
Container(  
  width: 100,  
  decoration: BoxDecoration(border: Border.all()),  
  child: Text(  
    overflow: TextOverflow.ellipsis,  
    'Hello $_name, how are you?',  
    textDirection: TextDirection.ltr,),  
  )  
)
```

Hello Ali, ...

Text

```
const Text.rich(  
  TextSpan(  
    text: 'Hello', // default text style  
    children: <TextSpan>[  
      TextSpan(  
        text: ' beautiful ',  
        style: TextStyle(  
          fontStyle: FontStyle.italic)  
        ),  
      TextSpan(  
        text: 'world',  
        style: TextStyle(  
          fontWeight: FontWeight.bold)  
        ),  
    ],  
  ),  
)
```

Hello, *beautiful* **world**

TextField

A text field lets the user enter text, either with hardware keyboard or with an onscreen keyboard.



Enter a search term

TextField

- InputDecoration includes label, icon, inline hint text, and error text.
- To remove the decoration entirely, set the decoration to null.

```
TextField(  
  decoration: InputDecoration(  
    border: OutlineInputBorder(),  
    hintText: 'Enter a search term',  
  ), ),
```



TextField

- You can get input value from a TextField in 3 ways:
 - `onChanged()`
 - `onSubmitted()`
 - Controller

Reading Values

A common way to read a value from a TextField is to use the **onSubmitted** callback.

This callback is applied to the text field's current value when the user finishes editing.

It can be used to manually move focus to another input widget when a user finishes with the currently focused input widget.

Reading Values

The default behavior of getting value is an invocation of **onChanged**.

The text field calls the **onChanged** callback whenever the user changes the text in the field.

TextField

```
TextField(  
  decoration: InputDecoration(  
    labelText: 'Email Address',  
    hintText: 'Enter your email',  
    icon: Icon(Icons.email),  
  ),  
  keyboardType: TextInputType.emailAddress,  
  
  // Reference to a TextEditingController object  
  controller: _emailController,  
  onChanged: (text) => print('Email: $text'),  
),
```



Email Address

Text Fields

Selecting the appropriate keyboard type, you can significantly improve the user's input experience and reduce errors.



text

Normal text keyboard



emailAddress

Normal text keyboard
with the @ character



number

Basic number keypad

phone

Phone-style keypad

TextField

The `TextEditingController` class is used to manage the text input in a `TextField`.

It allows you to:

1. Set the initial text: Set the default text that appears in the `TextField`.
2. Retrieve the current text: Get the text that the user has entered.
3. Listen to text changes: Respond to changes in the text as the user types.
4. Clear the text: Remove all text from the `TextField`.

TextField

```
class _MyTextFieldAppState extends State<MyTextFieldApp> {  
  final TextEditingController _textController = TextEditingController();
```

```
  void _submitText() {  
    String text = _textController.text;  
    // Do something with the text, e.g., print it to the console  
    print("Entered text: $text");  
  }  
  ...
```

Retrieve the current text

```
  TextField(  
    controller: _textController,  
    hintText: 'Enter your text here',),
```

Assign controller

```
  ElevatedButton(  
    onPressed: _submitText,  
    child: Text('Submit'),),
```

TextField

Clear the text.

```
_textController.clear();
```

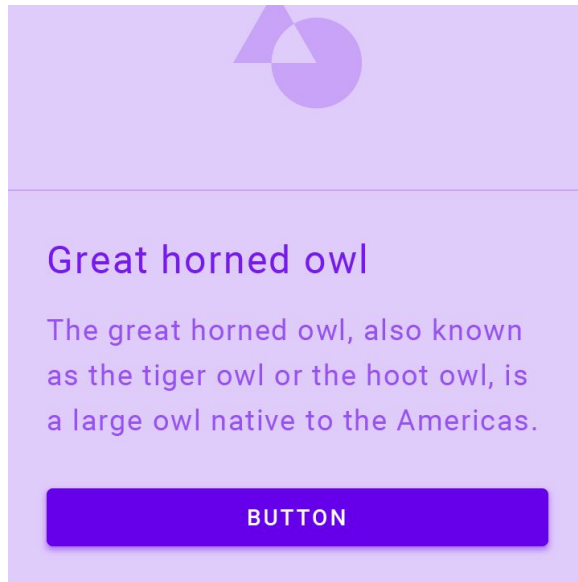
Dispose the Controller when the widget is removed to avoid memory leaks.

```
@override  
void dispose() {  
  _textController.dispose();  
  super.dispose();  
}
```

ElevatedButton

Creates buttons for user interaction.

A filled button whose material elevates when pressed.

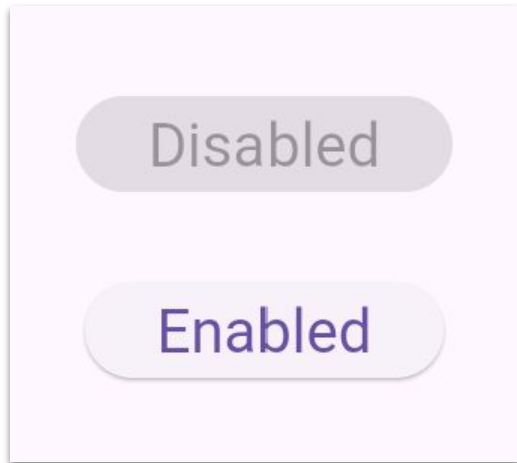


ElevatedButton

```
ElevatedButton(  
  style: style,  
  onPressed: null,  
  child: const Text('Disabled'),  
),
```

```
const SizedBox(height: 30),
```

```
ElevatedButton(  
  style: style,  
  onPressed: () {},  
  child: const Text('Enabled'),  
),
```



Elevated Button

Text Button

Outlined Button



IconButton



FloatingActionButton

Image

Image: Displays images.

Ways to display images:

1. Assets folder
2. Explicit device's directory
3. Network



Image

1. Assets folder

Step 1:

Create a folder named '**assets**' in the project **root directory**

Step 2:

Copy all images to the assets folder.

Reference: <https://api.flutter.dev/flutter/widgets/Image/Image.asset.html>

Image

1. Assets folder

Step 3:

Open the project's **pubspec.yaml** file and enter the image file names

```
flutter:  
  assets:  
    - images/cat.png  
    - images/2x/cat.png  
    - images/3.5x/cat.png
```

Image

1. Assets folder

Step 4:

Write code to display an image from asset

```
Image.asset('images/cat.png')
```

Image

2. Explicit device's directory

Displays an image obtained from a specific location, e.g. from a photo app

```
Image.file(selectedImages, ),
```

Reference: <https://api.flutter.dev/flutter/widgets/Image/Image.file.html>

Image

3. Network

To load images from a network URL:

```
Image.network('https://example/image_name.jpg')
```

Note: Reading a file from network involve an asynchronous task, which cannot provide a result immediately when it is started

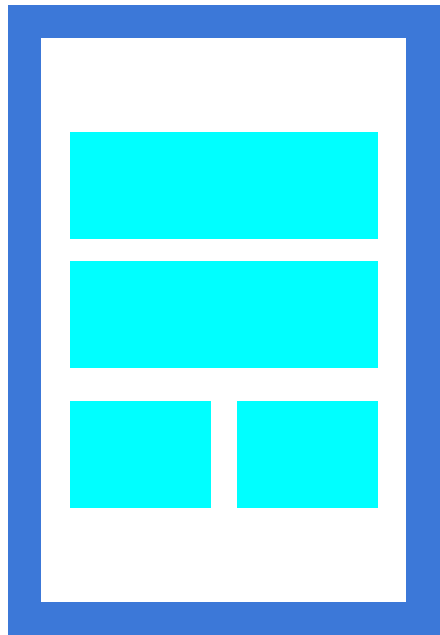
Image

Format image using

```
Image.asset(  
  file: file_name,  
  height: 100,  
  width: 100,  
  fit: BoxFit.fill,  
  alignment: Alignment.center,  
),
```

Row/Column

Arranges widgets horizontally (column) or vertically (row) for layout purposes.



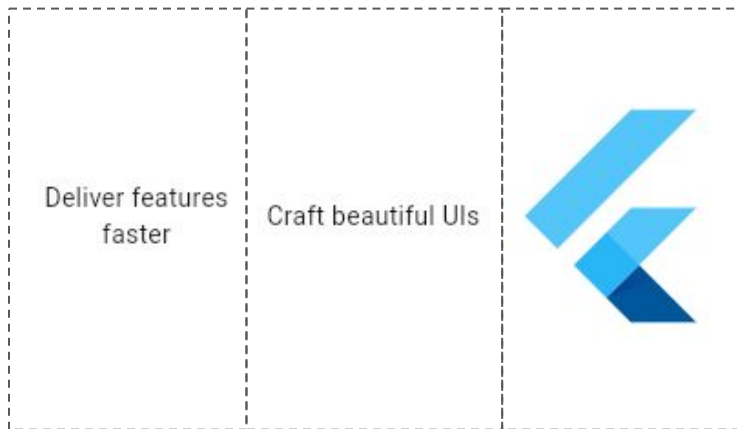
Row/Column

```
const Column(  
  children: <Widget>[  
    Text('Deliver features faster'),  
    Text('Craft beautiful UIs'),  
    Expanded(  
      child: FittedBox(  
        child: FlutterLogo(),  
      ),  
    ),  
  ],  
)
```



Row/Column

```
const Row(  
  children: <Widget>[  
    Expanded(  
      child: Text('Deliver features faster',  
        textAlign: TextAlign.center),  
    ),  
    Expanded(  
      child: Text('Craft beautiful UIs',  
        textAlign: TextAlign.center),  
    ),  
    Expanded(  
      child: FittedBox(  
        child: FlutterLogo(),  
      ),  
    ),  
  ],  
)
```



Icon

Icon: Displays icons.

```
const Row(  
  mainAxisAlignment: MainAxisAlignment.spaceAround,  
  children: <Widget>[  
    Icon(  
      Icons.favorite,  
      color: Colors.pink,  
      size: 24.0,  
      semanticLabel: 'Text to announce in accessibility modes',  
    ),  
    Icon(  
      Icons.audiotrack,  
      color: Colors.green,  
      size: 30.0,  
    ),  
    ...
```



Scaffold

Implements the basic Material Design visual layout structure.

It provides APIs for showing drawers, snack bars, and bottom sheets.

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('Sample Scaffold'),  
  ),  
  body: Center(...),  
);
```

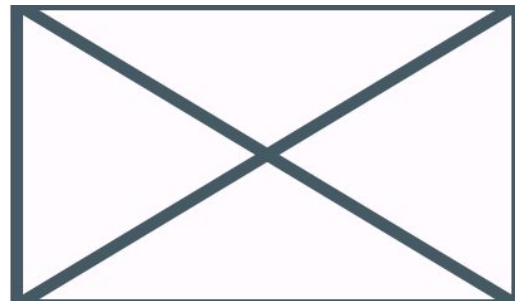


Placeholder

It represents a place where other widgets will be added later.

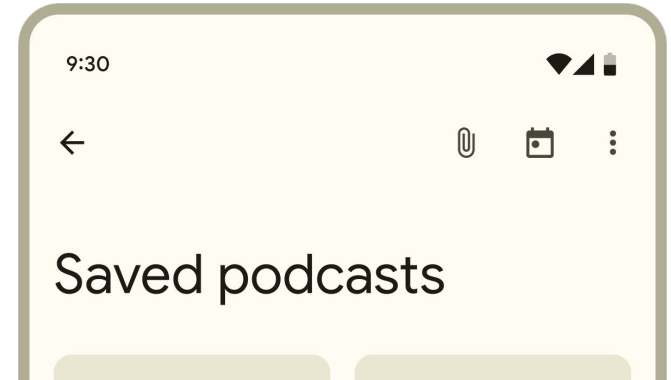
It is useful during development to indicate that the interface is not yet complete.

```
const Placeholder(  
  fallbackHeight: 30.0,  
  fallbackWidth: 100.0,  
  strokeWidth: 5.0,  
),
```



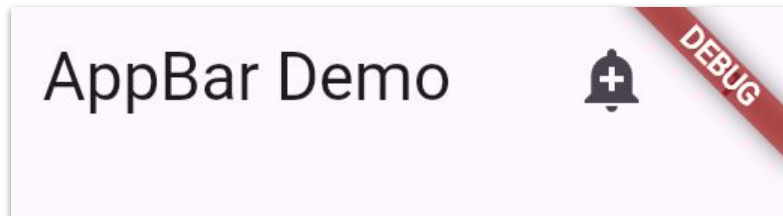
AppBar

Container that displays content and actions at the top of a screen.



AppBar

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('AppBar Demo'),  
      actions: <Widget>[  
        IconButton(  
          icon: const Icon(Icons.add_alert),  
          tooltip: 'Show SnackBar',  
          onPressed: null,  
        ),  
      ],  
    ),  
  );  
}
```



Stateless vs Stateful Widget

Widgets - Stateless vs Stateful

Stateless widgets

Simpler and have a fixed appearance and behavior

Don't hold any internal state

Stateful widgets

Maintain their own state, which react to user interactions or changes in data

Manage state through a State object

Stateless Widget

```
class GreenFrog extends StatelessWidget {  
  const GreenFrog({ super.key });  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(color: const Color(0xFF2DBD3A));  
  }  
}
```

Stateless Widget

```
class Frog extends StatelessWidget {  
  const Frog({  
    super.key,  
    this.color = const Color(0xFF2DBD3A),  
    this.child,  
  });  
  
  final Color color;  
  final Widget? child;  
  
  @override  
  Widget build(BuildContext context) {  
    return ColoredBox(color: color, child: child);  
  }  
}
```

Stateful Widget

```
class YellowBird extends StatefulWidget {  
  const YellowBird({ super.key });  
  
  @override  
  State<YellowBird> createState() => _YellowBirdState();  
}  
  
class _YellowBirdState extends State<YellowBird> {  
  @override  
  Widget build(BuildContext context) {  
    return Container(color: const Color(0xFFFFE306));  
  }  
}
```

Stateful Widget

```
class Bird extends StatefulWidget {  
  const Bird({  
    super.key,  
    this.color = const Color(0xFFFFE306),  
    this.child,  
  });  
  
  final Color color;  
  final Widget? child;  
  
  @override  
  State<Bird> createState() =>  
    _BirdState();  
}
```

```
class _BirdState extends State<Bird> {  
  double _size = 1.0;  
  
  void grow() {  
    setState(() { _size += 0.1; });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: widget.color,  
      transform:  
        Matrix4.diagonal3Values(_size,  
                                _size, 1.0),  
      child: widget.child,  
    );  
  }  
}
```

Design Systems

Design Systems

Cupertino

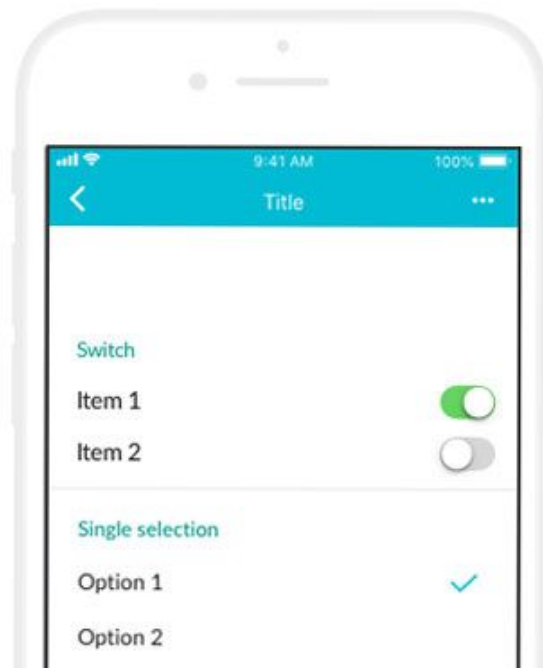
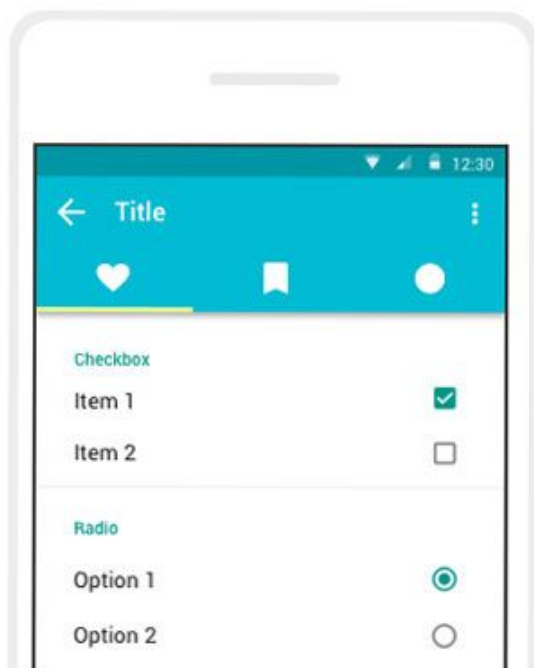
Widgets align with Apple's Human Interface Guidelines for iOS and macOS



Material Components

Visual, behavioral, and motion-rich widgets implementing the Material 3 design specification for Android, web, and desktop applications.



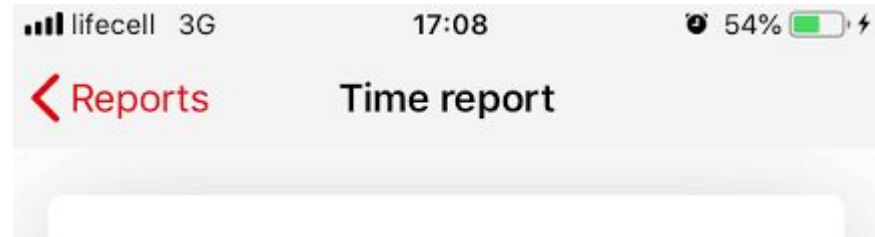


Cupertino Widgets

CupertinoPageScaffold: Provides a basic structure for a Cupertino-style app.

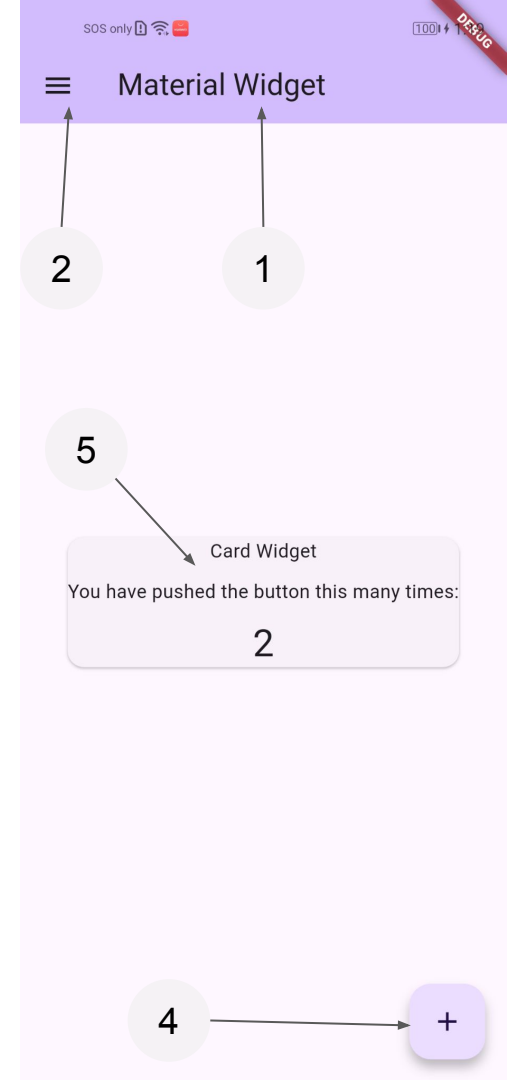
CupertinoNavigationBar: Creates a navigation bar for Cupertino-style apps.

CupertinoButton: Creates a button with a Cupertino-style appearance.



Material Design Widgets

1. AppBar: Creates an app bar at the top of the screen.
2. Drawer: Creates a side menu that slides in from the side.
3. Scaffold: Provides a basic structure for a material design app.
4. FloatingActionButton: Creates a circular button that floats above the content.
5. Card: Displays a rectangular card with a shadow.



Layout

Layout

- The core of Flutter's layout mechanism is widgets
- A layout is used to build more complex widgets



CALL



ROUTE



SHARE

Layout

- A layout is a widget.
- Two main categories of layout widgets:



Single-child



Multi-child

Single-child layout widgets

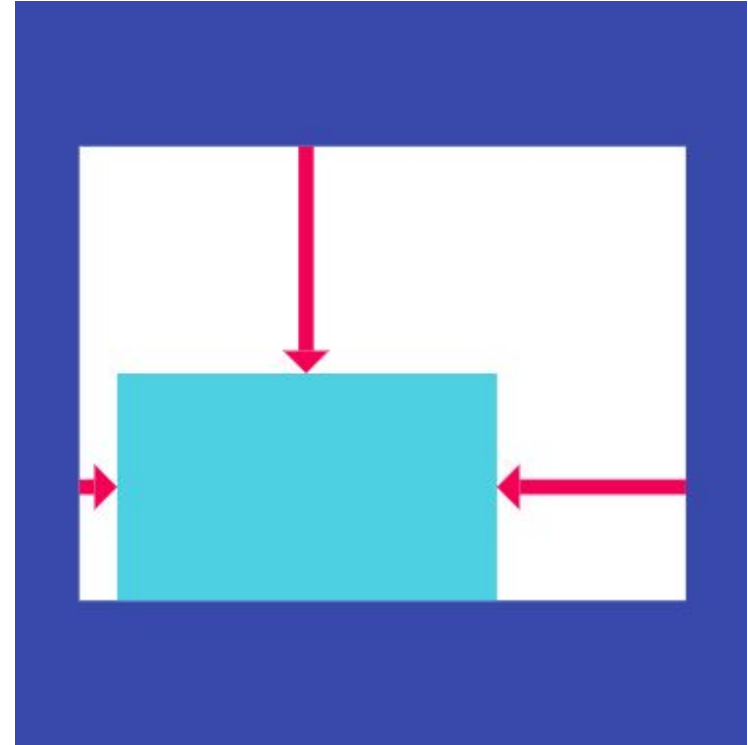
Single-child layout widgets

- These widgets can only hold one child widget.
- They are commonly used for tasks like:
 - Align
 - AspectRatio
 - Center
 - ConstrainedBox
 - Container
 - Padding
 - SizedBox



Align

A widget that aligns its child within itself and optionally sizes itself based on the child's size.



Align

```
Center(  
  child: Container(  
    height: 120.0,  
    width: 120.0,  
    color: Colors.blue[50],  
    child: const Align(  
      alignment: Alignment.topRight,  
      child: FlutterLogo(  
        size: 60,  
      ),  
    ),  
  ),  
)
```

Alignment.topRight



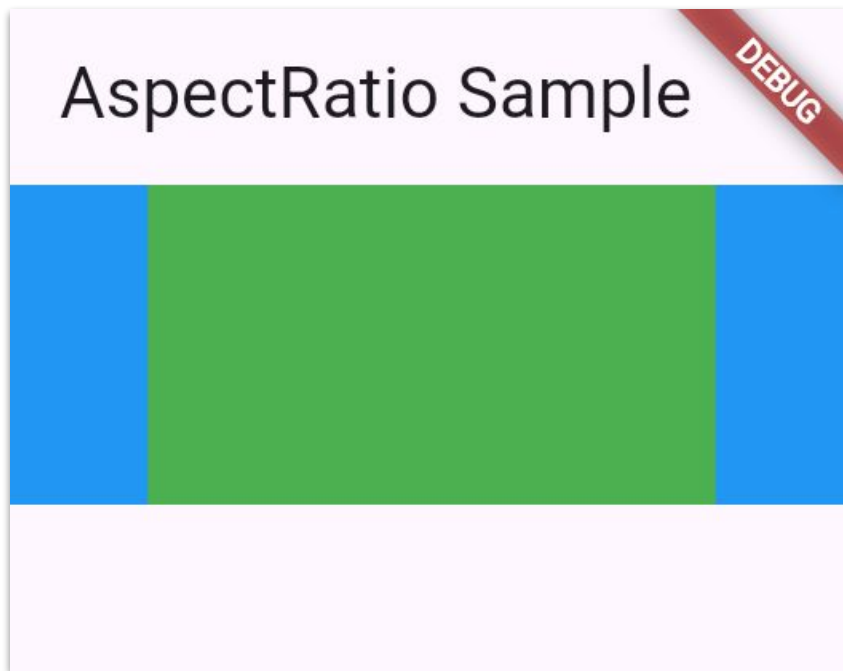
AspectRatio

A widget that attempts to size the child to a specific aspect ratio.



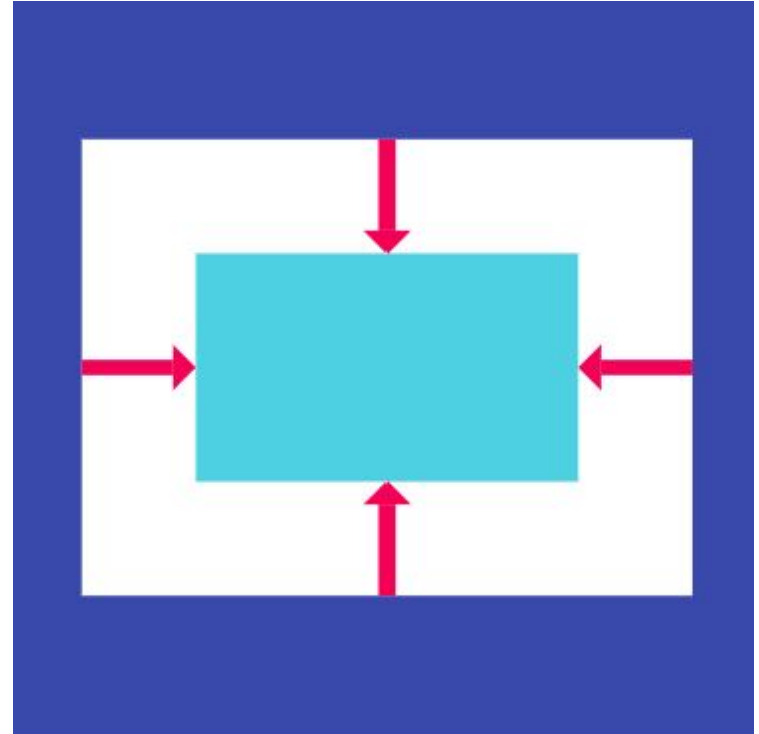
AspectRatio

```
Container(  
  color: Colors.blue,  
  alignment: Alignment.center,  
  width: double.infinity,  
  height: 100.0,  
  child: AspectRatio(  
    aspectRatio: 16 / 9,  
    child: Container(  
      color: Colors.green,  
    ),  
  ),  
)
```



Center

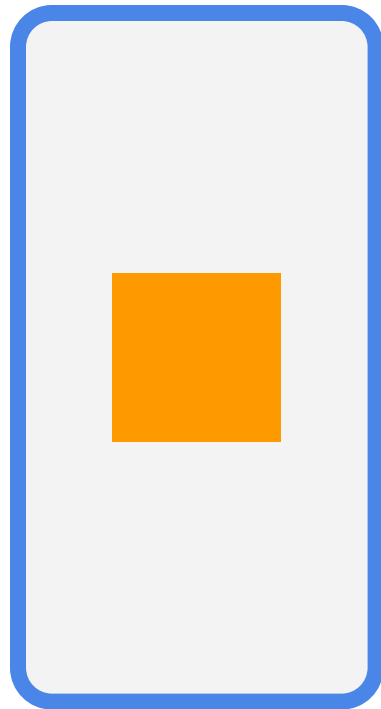
Alignment block that centers its child within itself.



Container

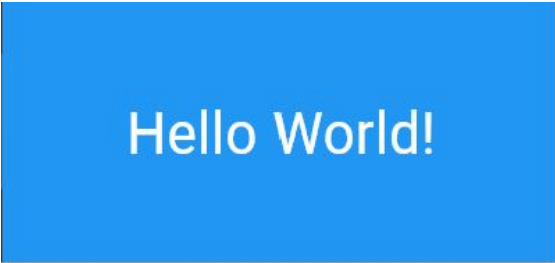
Groups other widgets and provides styling options like padding, margins, and borders.

```
Center(  
  child: Container(  
    margin: const EdgeInsets.all(10.0),  
    color: Colors.amber[600],  
    width: 48.0,  
    height: 48.0,  
  ),  
)
```



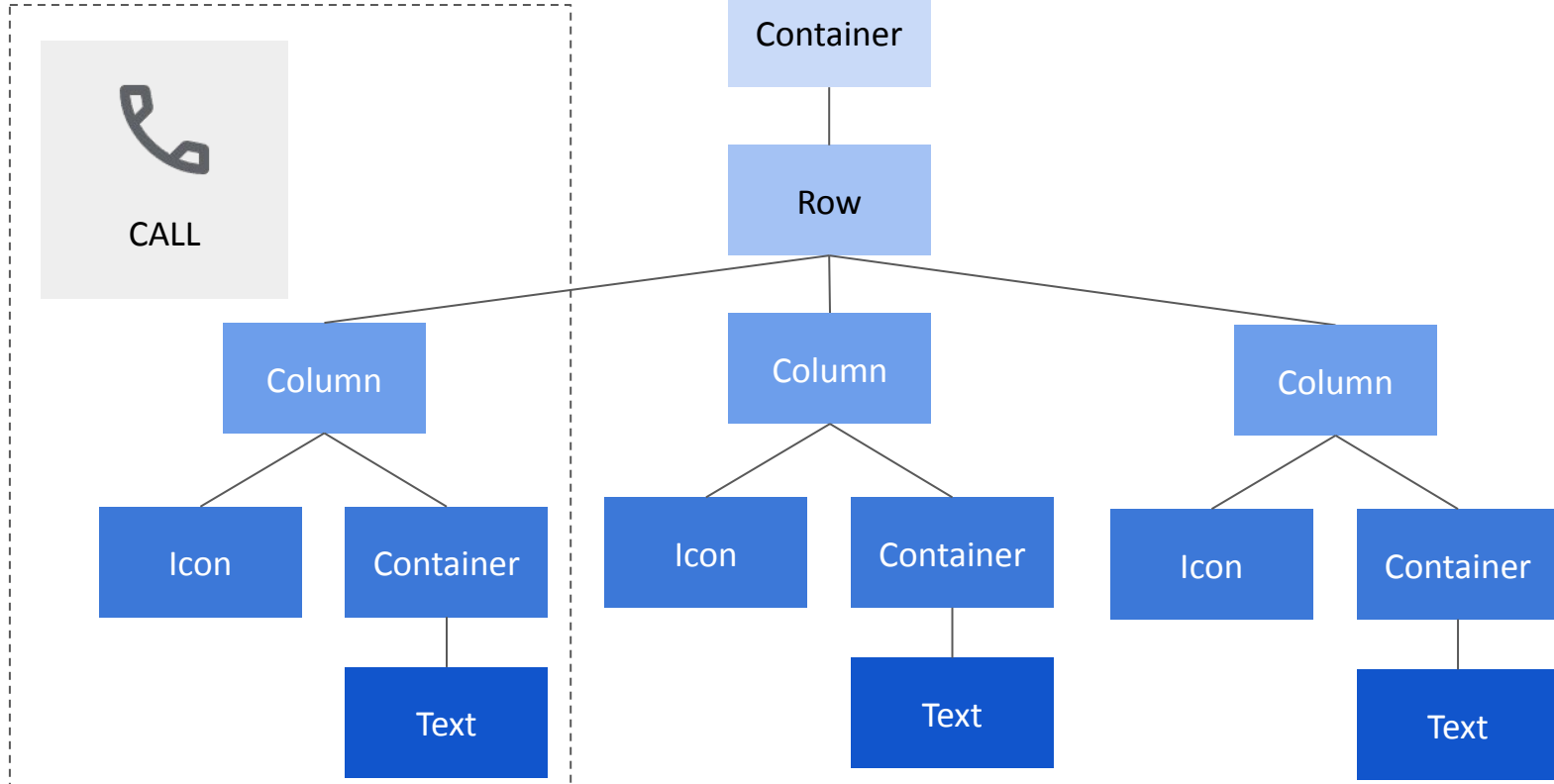
Container

```
Container(  
  width: 200.0,  
  height: 100.0,  
  color: Colors.blue,  
  padding: EdgeInsets.all(20.0),  
  alignment: Alignment.center,  
  child: Text(  
    "Hello World!",  
    style: TextStyle(  
      color: Colors.white,  
      fontSize: 20.0,  
      ...
```



Hello World!

Container



Padding

A widget that insets its child by the given padding.

```
const Card(  
  child: Padding(  
    padding: EdgeInsets.all(16.0),  
    child: Text('Hello World!'),  
  ),  
)
```



SizedBox

A box with a specified size. If given a child, this widget forces it to have a specific width and height.

```
const SizedBox(  
  width: 200.0,  
  height: 300.0,  
  child: Card(child: Text('Hello World!')), //optional  
)
```

Multi-child layout widgets

Multi-child layout widgets

These widgets can accommodate multiple child widgets and arrange them in different ways.

Common Multi-child Widgets:

- Row/Column
- GridView
- ListView
- Stack
- Wrap



Multi-child layout widgets

Key Features:

1. Layout Management

These widgets handle the arrangement and positioning of their child widgets within the available space.

2. Alignment

You can control how child widgets are aligned within the parent widget using properties like **mainAxisAlignment** (for Row and Column) or **alignment** (for Stack).

Multi-child layout widgets

Key Features:

3. Spacing (Flutter 3.27)

Properties like `mainAxisAlignment` (Row/Column) and `spacing` (Row/Column) allow you to control the spacing between child widgets.

4. Sizing

Some multi-child widgets offer options to control the size of their child widgets, such as **`mainAxisSize`** (Row/Column) or `flex` (Flex).

Row and Column

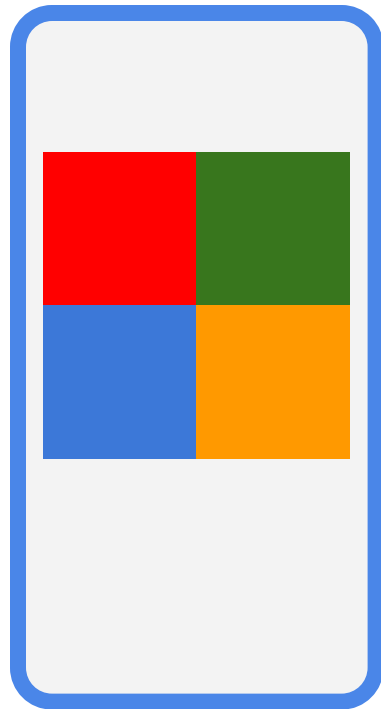
```
Row(  
    children:[  
        Text('Widget 1'),  
        Text('Widget 2'),  
        Text('Widget 3'),  
    ],  
)
```

```
Column(  
    children:[  
        Text('Widget 1'),  
        Text('Widget 2'),  
        Text('Widget 3'),  
    ],  
)
```


GridView

A scrollable, 2D array of widgets.

```
GridView.count(  
  crossAxisCount: 2,  
  children: [  
    Container(color: Colors.red,),  
    Container(color: Colors.green,),  
    Container(color: Colors.blue,),  
    Container(color: Colors.orange,),  
  ],  
),
```

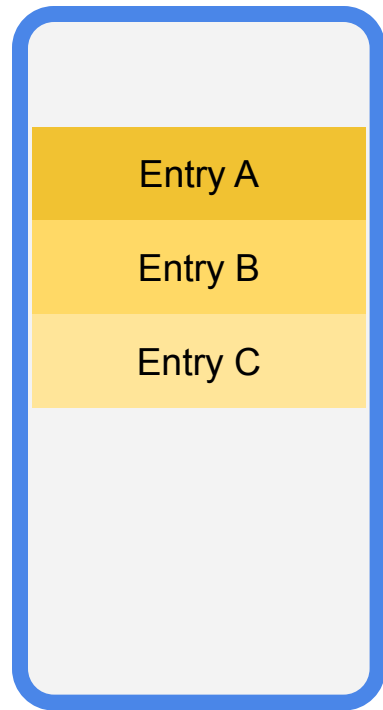


ListView

A scrollable list of widgets arranged linearly.

Suitable for lists that contain only a few items.

```
body: ListView(  
  children: [  
    Container(height: 50, color: Colors.amber[600],  
      child: const Center(child: Text('Entry A')),  
    ),  
    Container(height: 50, color: Colors.amber[500],  
      child: const Center(child: Text('Entry B')),  
    ),  
    Container(height: 50, color: Colors.amber[100],  
      child: const Center(child: Text('Entry C')),  
    ),  
  ],  
),
```

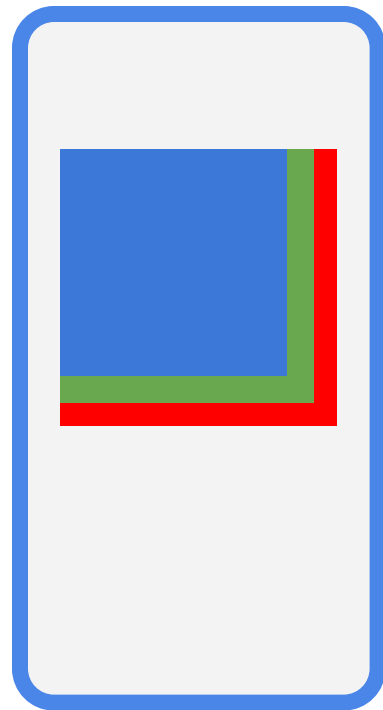


Stack

A widget that positions its children relative to the edges of its box.

It is useful if you want to overlap several children in a simple way.

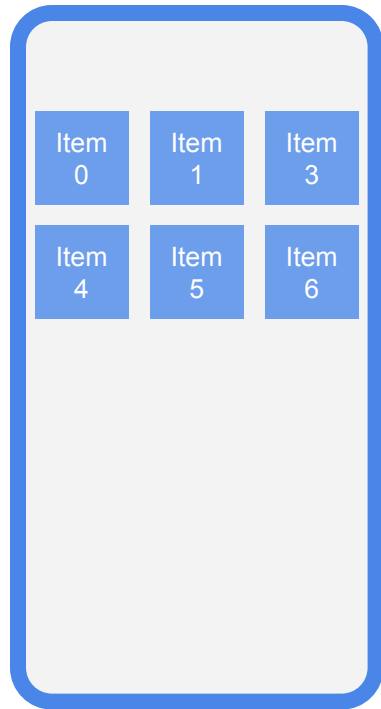
```
Stack(  
  children: <Widget>[  
    Container(width: 100, height: 100,  
      color: Colors.red,),  
    Container(width: 90, height: 90,  
      color: Colors.green,),  
    Container(width: 80, height: 80,  
      color: Colors.blue,),  
  ],  
)
```



Wrap

A widget that displays its children in multiple horizontal or vertical runs.

```
Wrap(  
  children: List.generate(6, (index) {  
    return Padding(  
      padding: const EdgeInsets.all(8.0),  
      child: Container( width: 100, height: 100,  
        color: Colors.lightBlue,  
        child: Center(  
          child: Text('Item $index',)),),  
    );  
  }  
),
```



Form Widgets

TextField: Allows users to enter text.

Checkbox: Creates a checkbox for user input.

RadioListTile: Creates a radio button for user input.

DropDownButtonFormField: Creates a dropdown menu for selecting a value.

The image shows a mobile application interface for a 'Signup Form'. The top status bar indicates 'Operator' with a Wi-Fi icon and the time '5:00 p.m.'. The app's header is blue with a back arrow and the title 'Signup Form'. The form itself is on a light gray background and contains four text input fields: 'Full Name', 'Email', 'Password', and 'Confirm Password'. A green checkmark is visible to the right of the 'Confirm Password' field. Below the fields is a toggle switch for the text 'I have read and accept the terms of service.'. At the bottom of the form is a blue button with the text 'Signup'.

Layout Widgets

Flex: Provides more control over the layout of its children using flex factors.

Wrap: Wraps its children to the next line when the available space runs out.

CustomScrollView: Allows you to create custom scrolling experiences.

SliverAppBar: Creates an app bar that can expand, collapse, and display different content as the user scrolls.

Other Widgets

FutureBuilder: Builds a widget based on the result of an asynchronous computation.

StreamBuilder: Builds a widget based on the latest data from a stream.

GestureDetector: Detects gestures on a child widget.

Hero: Creates a hero animation between two widgets in different screens.

Sliver Widgets

Sliver widgets

- Sliver widgets are the building blocks for creating **custom scrolling experiences**.
- They provide a more **granular approach** to scroll view construction compared to standard widgets like *ListView* or *GridView*.

Sliver widgets

Concepts

- Unlike regular box widgets, slivers are designed specifically for **scrollable areas**.
- They can **dynamically adjust their size and shape** based on the viewport and scrolling position.
- This allows for creating intricate **scrolling effects** like parallax effects, expanding headers, and custom layouts within a scrollable view.

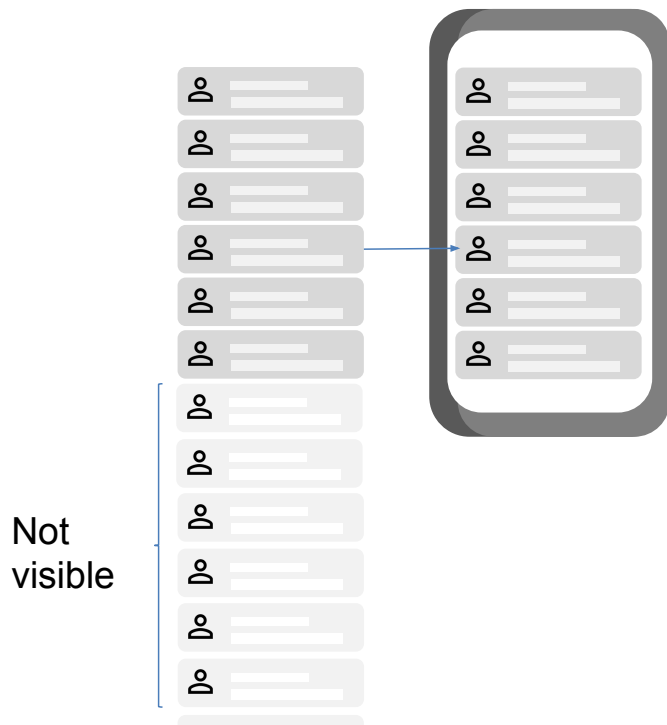
Sliver widgets

Benefits

- **Customization:** Sliver widgets offer fine-grained control over scrolling behavior and appearance.
- **Performance:** For large data sets, slivers can be more efficient than standard scrollable widgets.
- **Flexibility:** You can combine different slivers to create unique and engaging scrolling experiences.

Sliver widgets

All records are loaded to the ListView



Load records that are visible to the user using SilverList



Sliver widgets

Using Sliver Widgets

- Sliver widgets are typically used with the CustomScrollView widget.
- CustomScrollView takes a list of slivers as its slivers property, allowing you to combine different slivers to achieve your desired layout.

Sliver widgets

Common Sliver Widgets:

- **SliverAppBar**: Creates an app bar that can expand, collapse, and display different content as the user scrolls.
- **SliverList**: Renders a list of widgets vertically, similar to `ListView`, but with more flexibility in sizing and performance optimizations for large lists.
- **SliverGrid**: Renders a grid of widgets in a similar way to `GridView`, but within the context of a scrollable area.
- **SliverToBoxAdapter**: Embeds a regular box widget (like a `Text` or a `Container`) within the scrollable area.
- **SliverFixedExtentList**: Similar to `SliverList` but forces all its children to have the same height, improving rendering efficiency for uniform lists.

Performance Best Practices

Performance Best Practices

1. Minimize Widget Rebuilds

Avoid repetitive and costly work in **build()** methods since it can be invoked frequently when ancestor widgets rebuild.

Use **const** for **immutable widgets**: This tells Flutter that the widget's properties won't change, leading to optimizations.

Utilize **const constructors**: Use const constructors for widgets that don't require state.

Performance Best Practices

2. Optimize Layout

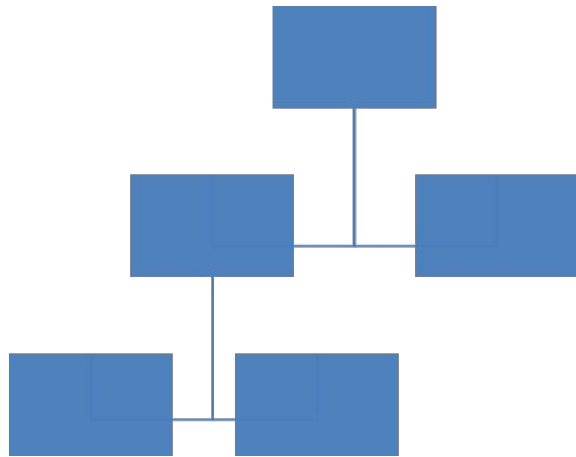
Avoid unnecessary nesting: Keep your widget tree as flat as possible to reduce rendering time.

Use **LayoutBuilder**: For dynamic layouts that depend on available space, use LayoutBuilder.

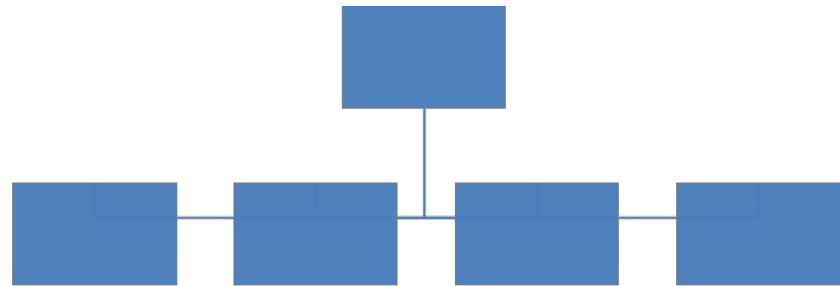
Performance Best Practices

2. Optimize Layout

Which layout is better in term of performance?



(A)



(B)

Performance Best Practices

3. Efficient Image Loading

Use `Image.network`: For network images, use `Image.network` with appropriate caching and loading strategies.

Optimize Image Size: Resize images to the appropriate size for your app.

Use efficient image formats like WebP, which is a raster graphics file format to replace JPEG, PNG and GIF.

Performance Best Practices

4. State Management

Use setState judiciously: Only call setState when necessary to trigger a rebuild.

Consider **state management** solutions: For complex state management, explore options like Provider, Riverpod (builds on the Provider package), or Business Logic Component (BLoC).

Performance Best Practices

5. Use Flutter's Build-in Widget

Use Flutter's optimized widgets whenever possible.

Find Out More

Widgets, <https://docs.flutter.dev/ui/widgets>

TextField, <https://docs.flutter.dev/cookbook/forms/text-input>

Layout, <https://docs.flutter.dev/ui/layout>

Chapter 3.2

Platforms, Themes, and Design for Everyone

Contents

Colors

Material Design and Themes

Design for Everyone

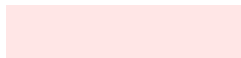
Colors

Colors

- Color and ColorSwatch constants which represent Material design color palette.
- Most swatches have colors from 100 to 900 in increments of one hundred, plus the color 50.
- The smaller the number, the more pale the color. The greater the number, the darker the color.
- The accent swatches (e.g. redAccent) only have the values 100, 200, 400, and 700.

Color Class

```
Color c1 = const Color(0xFFCE4EC);
```



```
Color c2 = const Color.fromARGB(0xFF, 0x42, 0xA5, 0xF5);
```

```
Color c3 = const Color.fromARGB(255, 66, 165, 245);
```

```
Color c4 = const Color.fromRGBO(66, 165, 245, 1.0);
```

Colors

Select a specific color from one of the swatches:

```
// Selects a mid-range green.  
Color selection = Colors.green[400]!;
```

Each ColorSwatch constant is a color and can used directly:

```
Container(  
    // same as Colors.blue[500] or Colors.blue.shade500  
    color: Colors.blue,  
)
```

ColorSwatch

ColorSwatch is a class that represents a collection of colors related to a single color theme.

It's useful for creating color palettes that vary in shades, tints, and tones of a base color.

```
ColorSwatch myColorSwatch =  
    MaterialColor(0xFF007AFF, {  
        50: Color(0xFF007AFF),  
        100: Color(0xFF007AFF),  
        200: Color(0xFF007AFF),  
        300: Color(0xFF007AFF),  
        400: Color(0xFF007AFF),  
        500: Color(0xFF007AFF),  
        'arrow': Color(0xFF20877E),  
        'border': Color(0xFF20877E),  
    });
```

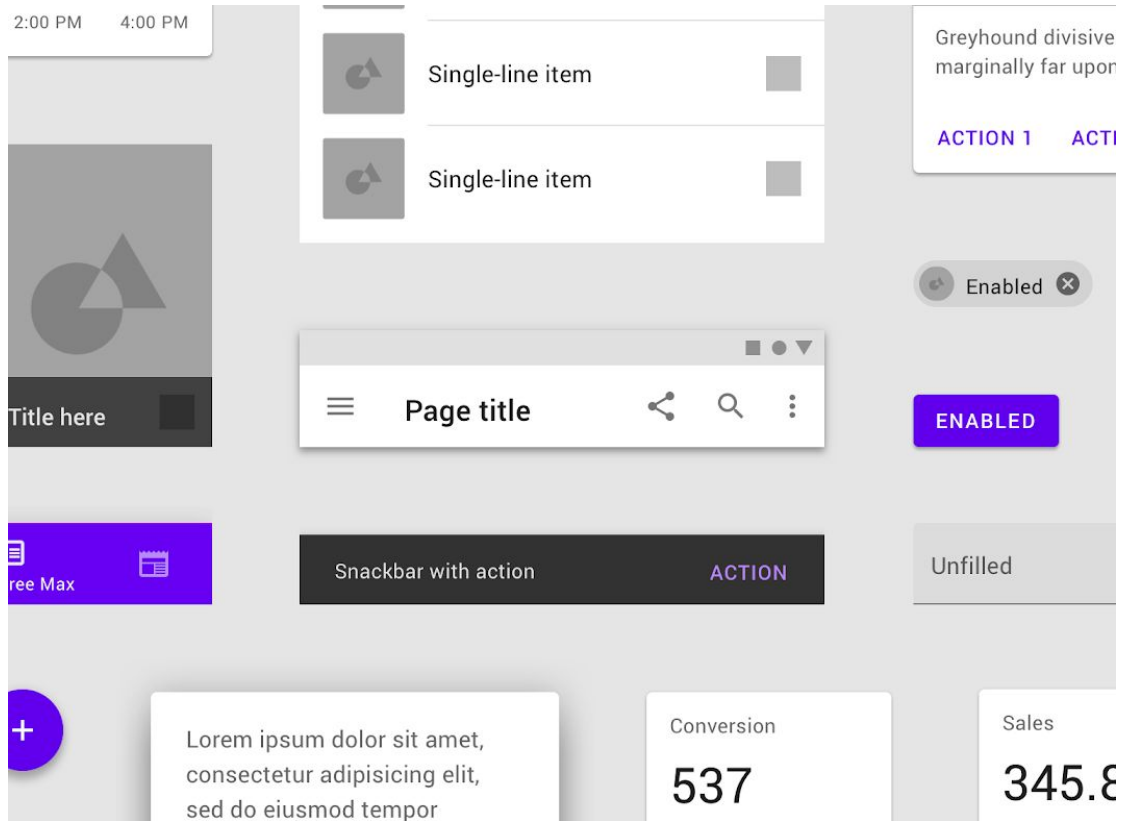
```
Container(  
    // Access a specific shade  
    color: myColorSwatch[500], )  
  
border: Border.all(  
    color: myColorSwatch['border'],  
),
```

Material Design and Themes

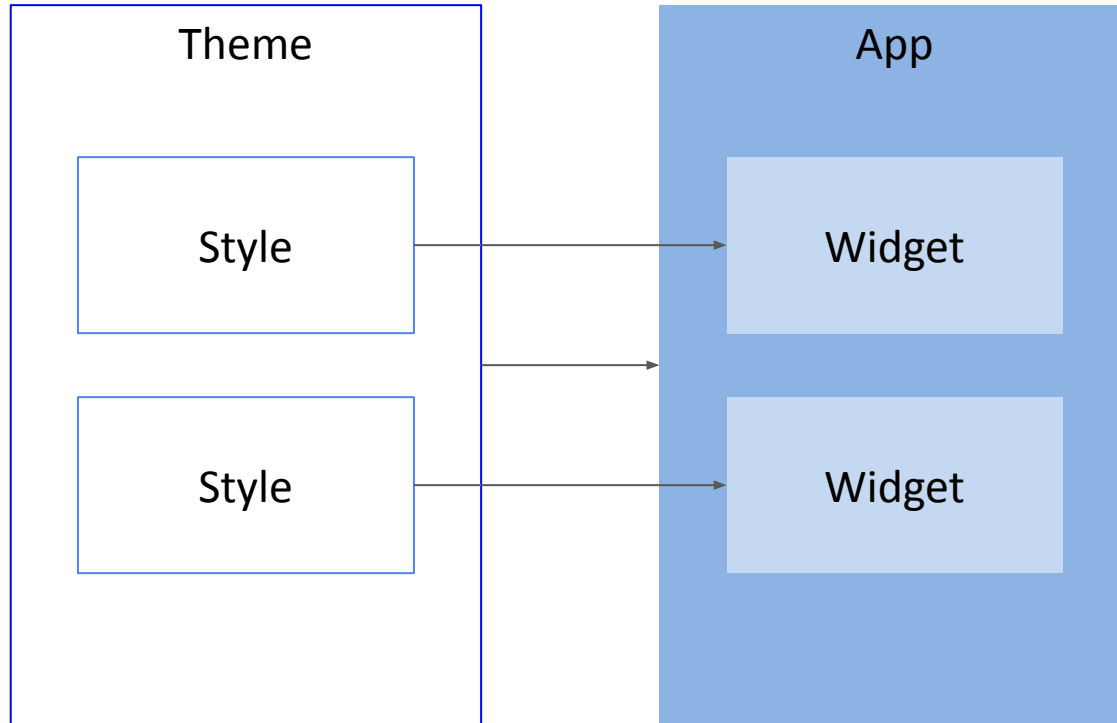
Material Design

Material Design is an open-source design system built and supported by Google designers and developers.

Default UI design for Flutter app.



Styles and Themes



Themes

To share colors and font styles throughout an app.

You can define app-wide themes.

You can extend a theme to change a theme style for one component.

Each theme defines the colors, type style, and other parameters applicable for the type of Material component.

Themes

Flutter applies styling in the following order:

1. Styles applied to the specific widget.
2. Themes that override the immediate parent theme.
3. Main theme for the entire app.

Themes

Most instances of ThemeData set values for the following two properties. These properties affect the entire app.

- colorScheme defines the colors.
- textTheme defines text styling.

```
MaterialApp(  
  title: appName,  
  theme: ThemeData(  
    useMaterial3: true,  
  
    // Define the default brightness and colors.  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: Colors.purple,  
      // ...  
      brightness: Brightness.dark,  
    ),  
  
    // Define the default `TextTheme`. Use this to specify the default  
    // text styling for headlines, titles, bodies of text, and more.  
    textTheme: TextTheme(  
      displayLarge: const TextStyle(  
        fontSize: 72,  
        fontWeight: FontWeight.bold,  
      ),  
      // ...  
      titleLarge: GoogleFonts.oswald(  
        fontSize: 30,  
        fontStyle: FontStyle.italic,  
      ),  
      bodyMedium: GoogleFonts.merriweather(),  
      displaySmall: GoogleFonts.pacifico(),  
    ),  
  ),  
),
```

Using GoogleFonts
from pub.dev package

```
Container(  
  padding: const EdgeInsets.symmetric(  
    horizontal: 12,  
    vertical: 12,  
  ),  
  color: Theme.of(context).colorScheme.primary,  
  child: Text(  
    'Text with a background color',  
    // ...  
    style: Theme.of(context).textTheme.bodyMedium!.copyWith(  
      color: Theme.of(context).colorScheme.onPrimary,  
    ),  
  ),  
),
```

Override a Theme

To override the overall theme in part of an app, wrap that section of the app in a **Theme** widget.

You can override a theme in two ways:

1. Create a unique **ThemeData** instance.
2. Extend the parent theme.

Set a unique ThemeData instance

If you want a component of your app to ignore the overall theme, create a ThemeData instance. Pass that instance to the Theme widget.

```
Theme(  
  // Create a unique theme with `ThemeData`.  
  data: ThemeData(  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: Colors.pink,  
    ),  
  ),  
  child: FloatingActionButton(  
    onPressed: () {},  
    child: const Icon(Icons.add),  
  ),  
);
```


Fonts and Typography

Typography covers the style and appearance of fonts.

Font Style:

1. **Typeface**: a set of common character rules. E.g. Roboto
2. **Size**: E.g. Size 18
3. **Style**: E.g. Regular, Italic
4. **Weight**: E.g. Normal, Bold

Fonts and Typography

TextTheme could be apply to:

- Display
- Headline
- Title
- Label
- Body

Size Variation:

- Small
- Medium
- Large

Display Large

Display Medium

Display Small

Headline Large

Headline Medium

Headline Small

Title Large

Title Medium

Title Small

Label Large

Label Medium

Label Small

Body Large

Body Medium

Body Small

Google Fonts Tester

<https://fonts.google.com/>

A

Fonts

🌐

Noto

🔍

Icons

📖

Knowledge

?

FAQ

 Google Fonts



A

Specimen

A

Type tester

A

Glyphs

i

About

📄

License

Get font

Roboto

Designed by [Christian Robertson](#)

Select preview text:

Writing system

Language

Heading

48px

Regular 400

I

A















Whereas recognition of the inherent dignity

No one shall be subjected to arbitrary arrest, detention or exile. Everyone is entitled in full equality to a fair and public hearing by an independent and impartial tribunal, in the

Serif vs Sans-Serif

Serif

Abc



Sans-Serif

Abc

Serif vs Sans-Serif

Which font type is easier to read?

Serif 30

Serif 24

Serif 18

Serif 14

Serif 12

Sans-Serif 30

Sans-Serif 24

Sans-Serif 18

Sans-Serif 14

Sans-Serif 12

Design for Everyone

Design for Everyone

Design for Everyone is a design philosophy that aims to create products and services that are **accessible** and **usable** by as many people as possible, regardless of their **age**, **ability**, or **background**.

In the context of **mobile app development**, it means making your app accessible to as many people as possible, regardless of their **ability**, **language**, or **device**.

Design for Everyone

Methods:

1. Internationalizing
2. Supporting different screens
3. Make apps more accessible

Internationalizing

Internationalizing

Internationalizing your app involves making it adaptable to different **languages**, **cultures**, and **regions**.

1. Language Translation
2. Generate Localization Delegates
3. Date, Number and Currency Formatting
4. Text Direction
5. Cultural Nuances

Setting up

Then import the `flutter_localizations` library and specify `localizationsDelegates` and `supportedLocales` for your `MaterialApp` or `CupertinoApp`:

```
import 'package:flutter_localizations/flutter_localizations.dart';  
import 'package:intl/intl.dart' as Intl;
```

Setting up

The `pubspec.yaml` file has the following entries:

```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_localizations:  
    sdk: flutter  
intl: any
```

Setting up

```
return const MaterialApp(  
  title: 'Localizations Sample App',  
  localizationsDelegates: [  
    GlobalMaterialLocalizations.delegate,  
    GlobalWidgetsLocalizations.delegate,  
    GlobalCupertinoLocalizations.delegate,  
  ],
```

```
  supportedLocales: [  
    Locale('en'), // English  
    Locale('es'), // Spanish  
  ],  
  home: MyHomePage(),  
);
```

Language Translation

Create .arb files (Android Resource Bundle) for each language.

Use the `generate_localized_json` package to generate Dart code from the .arb files.

```
// en.arb - English
{
  "helloWorld": "Hello, world!",
  "greeting": "Hello, {name}!"
}
```

```
//fr.arb - French
{
  "helloWorld": "Bonjour, le monde!",
  "greeting": "Bonjour, {name}!"
}
```

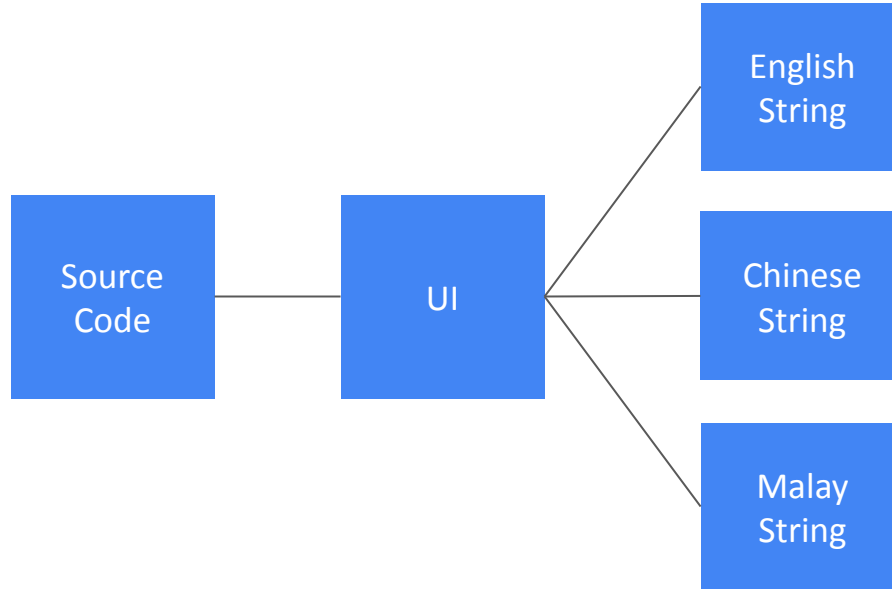
Language Translation

Mark translatable strings: Use `Intl.message()` to mark strings that need translation.

```
Text(Intl.message('Hello, world!')),
```

Create Locale Directories and String Files

At runtime, the system uses the appropriate set of string resources based on the locale currently set for the user's device.



Local Currency Symbol

```
final myCurrency = Intl.NumberFormat('#,##0.00', 'ms_MY');
```

```
symbol = myCurrency.currencySymbol;
```

```
output = myCurrency.format(total);
```

ISO 639 Language code

Format: [language code]_[country code]

Language	Language Code	Country Code	E.g.
US English	en	US	en_US
Malay (Malaysia)	ms	MY	ms_MY
Chinese (China)	zh	CN	zh_CN

ISO Language Code, https://en.wikipedia.org/wiki/List_of_ISO_639_language_codes

ISO Country Code, https://en.wikipedia.org/wiki/List_of_ISO_3166_country_codes

Supporting Different Screens

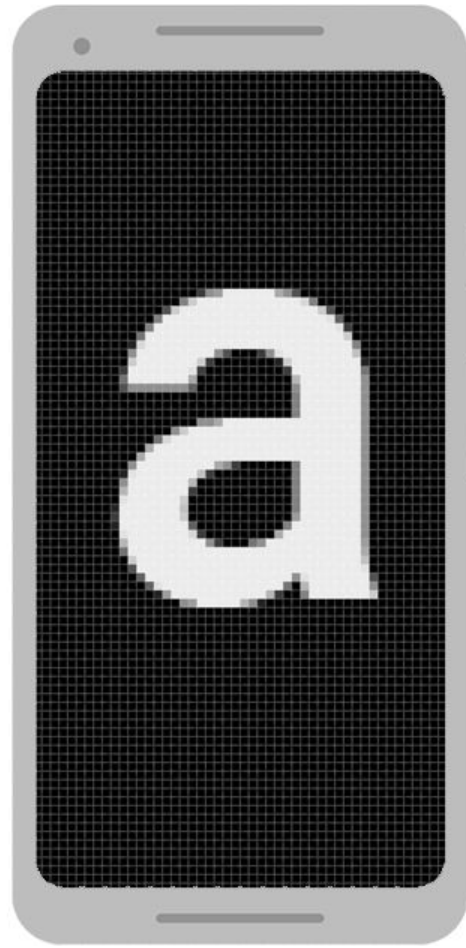
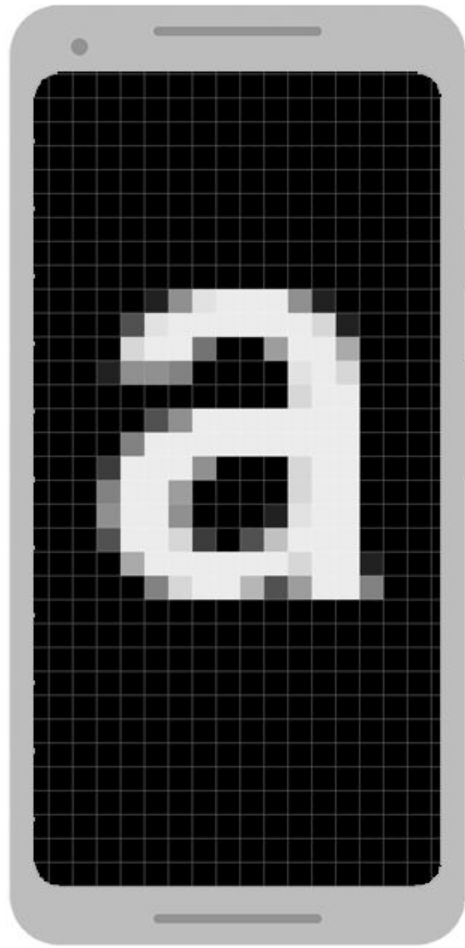
Supporting Different Screens

Categories of mobile device screens:

Size
Small
Normal
Large
xLarge

Density
Low (ldpi)
Medium (mdpi)
High (hdpi)
Extra high (xhdpi)





Supporting Different Screens

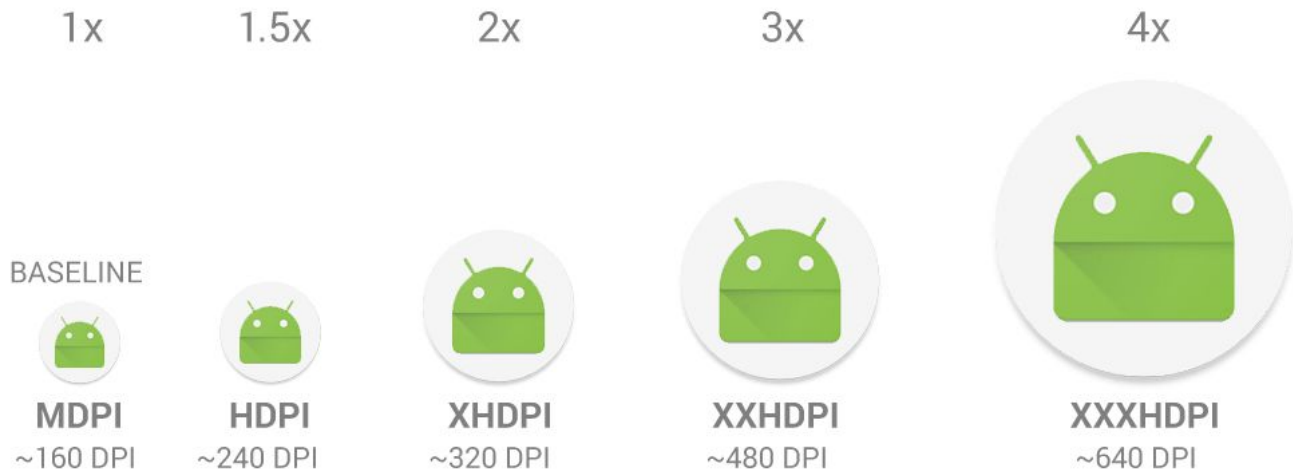
Adaptive Design

1. Utilize `Flexible` and `Expanded`: These widgets help you distribute space among child widgets.
2. Responsive Images: Use `FittedBox` to scale images to fit the available space.

Supporting Different Screens

Platform-Specific Considerations

In the context of Android and iOS, you can provide alternative bitmap resources for good graphical quality and performance



Supporting Different Screens

Create a directory named `assets` in your Flutter project's root directory.

Within this directory, create subfolders for each resolution: 1x, 2x, 3x, etc.

```
assets/  
  images/  
    logo.png  
    logo@2x.png  
    logo@3x.png
```

Supporting Different Screens

Add the `assets` section to your `pubspec.yaml` file:

```
flutter:  
  assets:  
    - assets/images/logo.png
```

Supporting Different Screens

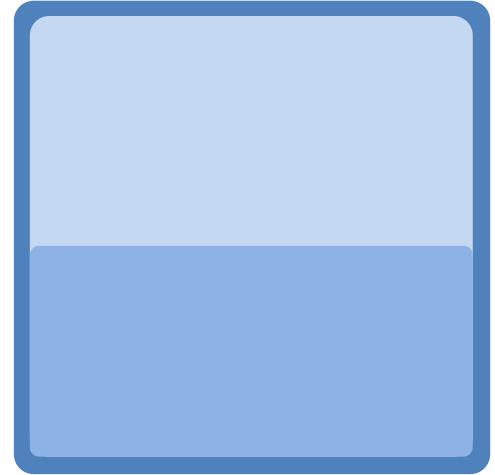
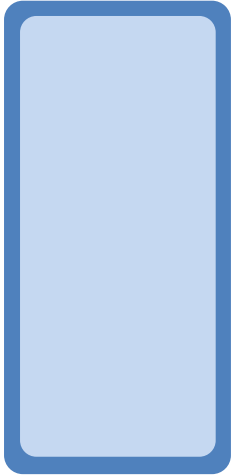
Use the `Image.asset()` widget to load the appropriate asset based on the device's screen density:

```
Image.asset('assets/images/logo.png'),
```

The system selects the most suitable asset based on the device's pixel density.

Supporting Different Screens

You should also consider screen orientation and slit screen.



Supporting Different Screens

You can build different layouts depending on a given Orientation.

```
import 'package:flutter/material.dart';

class MyWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: MediaQuery.of(context).orientation ==
        Orientation.portrait ? PortraitLayout() : LandscapeLayout(),
    );
  }
}
```

Supporting Different Screens

To force your app to a specific orientation, you can use the `SystemChrome` class:

```
void main() {  
  WidgetsFlutterBinding.ensureInitialized();  
  SystemChrome.setPreferredOrientations([  
    DeviceOrientation.portraitUp,  
  ]);  
  runApp(const MyApp());  
}
```

Support Display Cutouts



Tear
Drop



Notch



Punch Hole
(Middle)



Punch Hole
(Left)

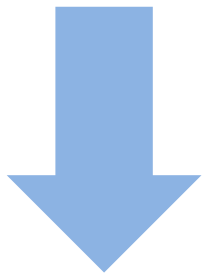


Punch Hole
(Right)

Support Display Cutouts



Specify Minimum and Target API Levels



minSdkVersion = the lowest API level with which your app is compatible



targetSdkVersion = the highest API level against which you've designed and tested your app

Specify Minimum and Target API Levels

In the Project settings:

```
android {  
    . . .  
  
    defaultConfig {  
        . . .  
        minSdkVersion 16  
        targetSdkVersion 34  
        . . .  
    }  
}
```



To support more Android devices

Specify Minimum and Target API Levels

In the Project settings:

```
android {  
    . . .  
  
    defaultConfig {  
        . . .  
        minSdkVersion 16  
        targetSdkVersion 34  
        . . .  
    }  
}
```



To include the latest OS features

Make Apps More Accessible

Accessibility

Regardless of ability, users are able to navigate, understand, and use an app successfully

Consideration:



Navigation



Readability



Guidance and Feedback

Accessibility - Navigation

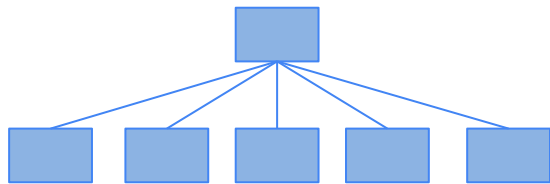
1. Support screen readers by Semantics.

```
Semantics(  
  label: 'This is a button',  
  child: ElevatedButton(  
    onPressed: () {},  
    child: Text('Click Me'),  
  ),  
),
```

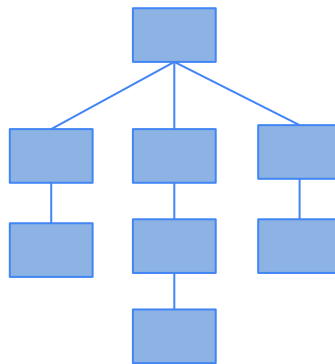
Accessibility - Navigation

2. Create easy-to-follow navigation

- Support keyboards or gestures input
- Avoid having UI elements fade out or disappear after a certain amount of time
- Create flat navigation structure



Do



Don't

Accessibility - Navigation

3. Make touch targets large

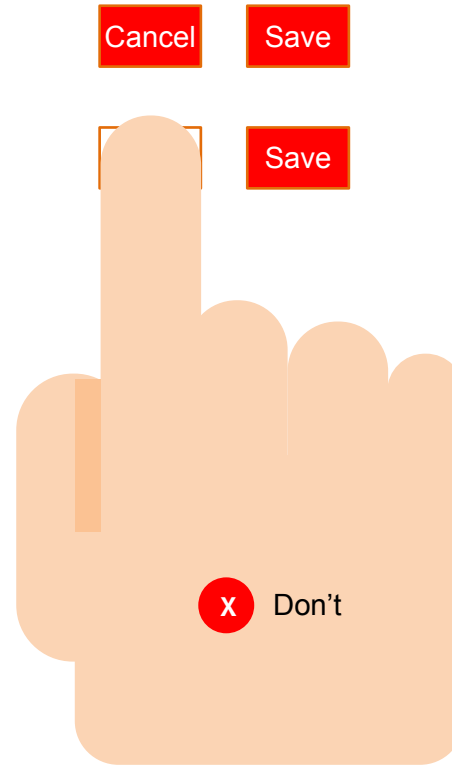
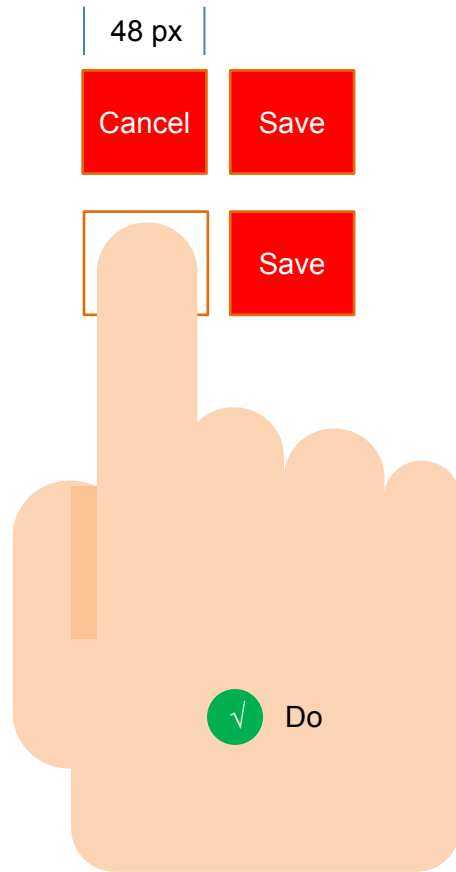
- Min size 48 x 48 pixel
- Space between elements min 8dp



Do

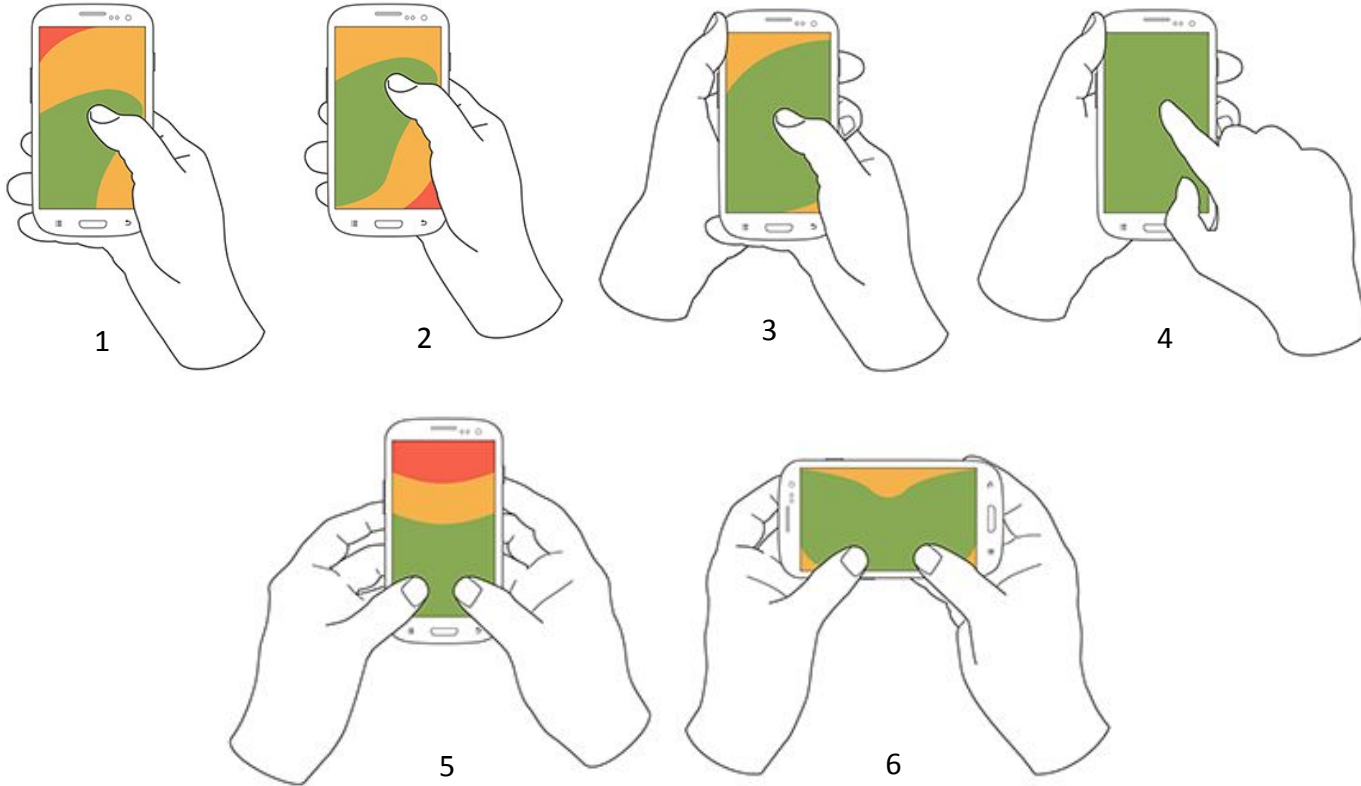


Don't

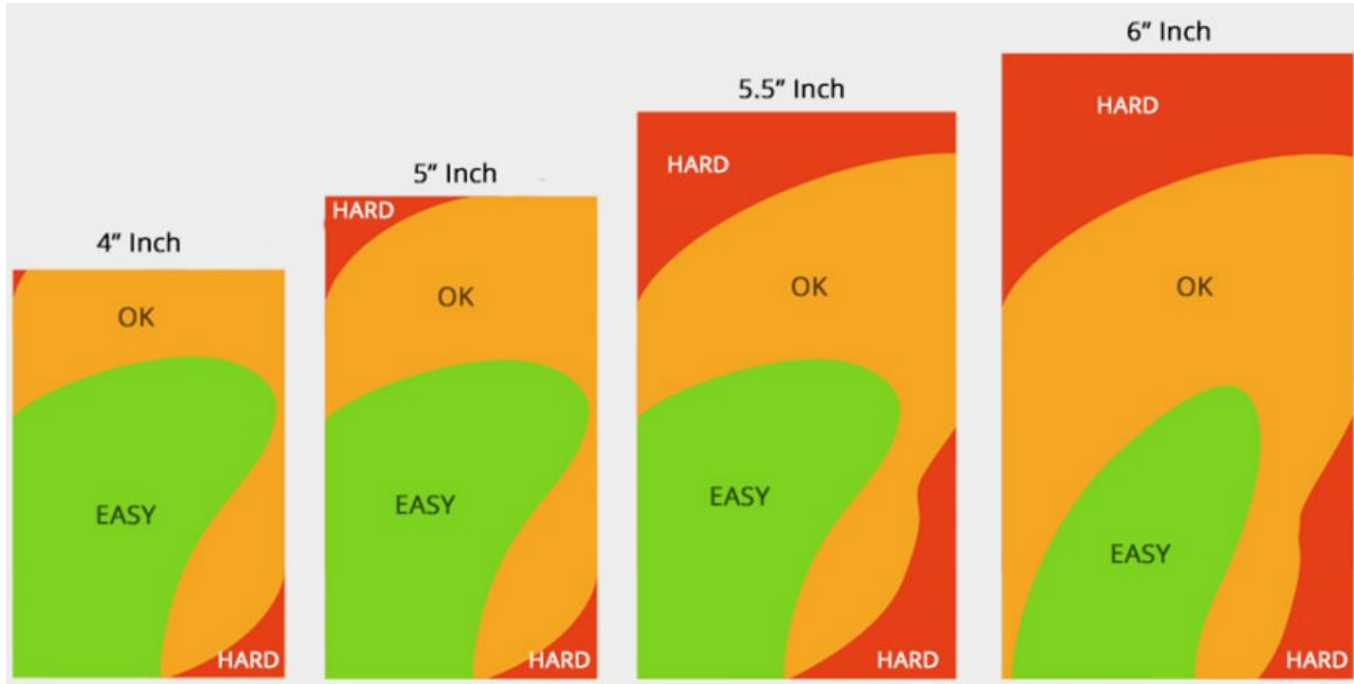


Index finger fits snugly inside.
Target edges give visual feedback.

How users hold their phones?



Touch Zone



<https://www.probeseven.com/blog/mobile-uiux-design-considerations-outdoors-visually-challenged/>

Gesture Navigation

Android 10 (API 29) supports fully gestured-based navigation

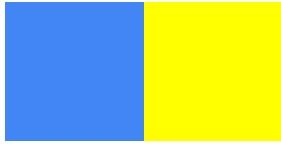
Ensure your apps:

- Extend content from edge to edge
- Handle conflicting gestures



Accessibility - Readability

4. Provide adequate colour contrast

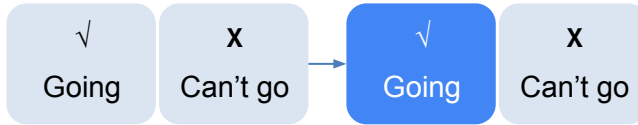


✓ Do



✗ Don't

Use more than just colour to convey information



✓ Do



✗ Don't

The Material Design Tools

<https://m2.material.io/design/color/the-color-system.html#tools-for-picking-colors>

Red 50	#FFEBEE
100	#FFCDD2
200	#EF9A9A
300	#E57373
400	#EF5350
500	#F44336
600	#E53935
700	#D32F2F
800	#C62828
900	#B71C1C
A100	#FF8A80

Pink 50	#FCE4EC
100	#F8BBD0
200	#F48FB1
300	#F06292
400	#EC407A
500	#E91E63
600	#D81B60
700	#C2185B
800	#AD1457
900	#880E4F
A100	#FF80AB

Purple 50	#F3E5F5
100	#E1BEE7
200	#CE93D8
300	#BA68C8
400	#AB47BC
500	#9C27B0
600	#8E24AA
700	#7B1FA2
800	#6A1B9A
900	#4A148C
A100	#EA80FC

Accessibility - Readability

5. Make media content more accessible

- Include controls for users to pause or stop video and audio files
- Provide transcript/caption

Accessibility - Guidance and Feedback

6. Make interactive controls clear and discoverable

- Interactive controls have text labels, tooltips, or placeholder text to indicate their purpose
- When naming elements, be consistent in your terminology throughout your app.

Find Out More

Colors, <https://api.flutter.dev/flutter/dart-ui/Color-class.html>

Themes, <https://docs.flutter.dev/cookbook/design/themes>

Internationalization,
<https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization>

Chapter 3.3

State Management and Navigation principles

Contents

State Management

Navigation Principles

Navigation

- Tab

- Drawer

- Dialog

- Picker

- Snackbar

State Management

State Management

State management involves managing the data that changes over time and triggers UI updates.

State of an app is everything that exists in memory when the app is running.

This includes the app's assets, all the variables that the Flutter framework keeps about the UI, animation state, textures, fonts, and so on.

State Management

Ephemeral State

Also called UI state or local state.

It is the state you can neatly contain in a single widget.

App State

Also called shared state

UI state that you want to share across many parts of your app, and that you want to keep between user sessions

State Management

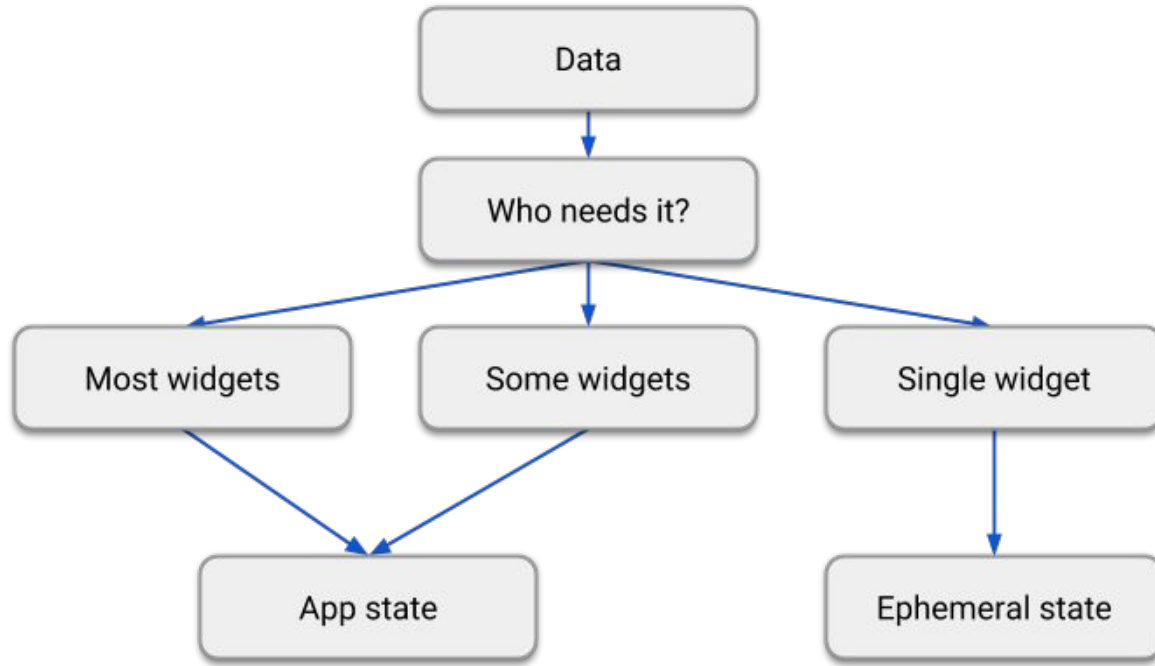
Examples of Ephemeral State:

- current page in a `PageView`
- current progress of a complex animation
- current selected tab in a `BottomNavigationBar`

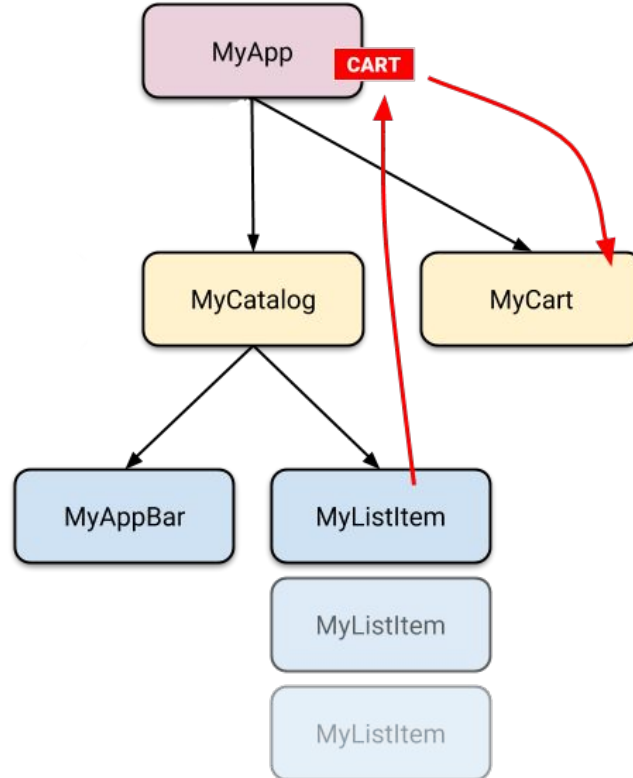
Examples of Application State:

- user preferences
- login info
- notifications in a social networking app
- the shopping cart in an e-commerce app
- read/unread state of articles in a news app

State Management



State Management



State Management - Provider

Provides a simple way to share state across multiple widgets.

Uses `ChangeNotifier` to notify listeners when the state changes.

dependencies:

flutter:

 sdk: flutter

provider:version

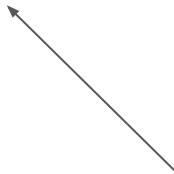
State Management - Provider

Insert dependency:

```
import 'package:provider/provider.dart';
```

State Management - Provider

```
class CounterProvider extends ChangeNotifier {  
  int _counter = 0;  
  int get counter => _counter;  
  void increment() {  
    _counter++;  
    notifyListeners();  
  }  
}
```



It notify listeners when
the state changes

State Management - Provider

```
void main() {  
  runApp(  
    ChangeNotifierProvider(  
      create: (context) => CounterProvider(),  
      child: someWidget,  
    ),  
  );  
}
```

State Management - Provider

Widget with multiple providers:

```
MultiProvider(  
  providers: [  
    ChangeNotifierProvider(create: (context) => Provider1()),  
    ChangeNotifierProvider(create: (context) => Provider2()),  
  ],  
  child: somewidget,  
),
```

State Management - Provider

```
body: Center(  
  child: Consumer<CounterProvider>(←  
    builder: (context, counter, child) {  
      return Text($counter.counter,);  
    },  
  ),  
,  
  
floatingActionButton: FloatingActionButton(  
  onPressed: () {  
    context.read<CounterProvider>().increment();  
  },  
  tooltip: 'Increment',  
  child: const Icon(Icons.add),  
)
```



Consumer widget

Navigation principles

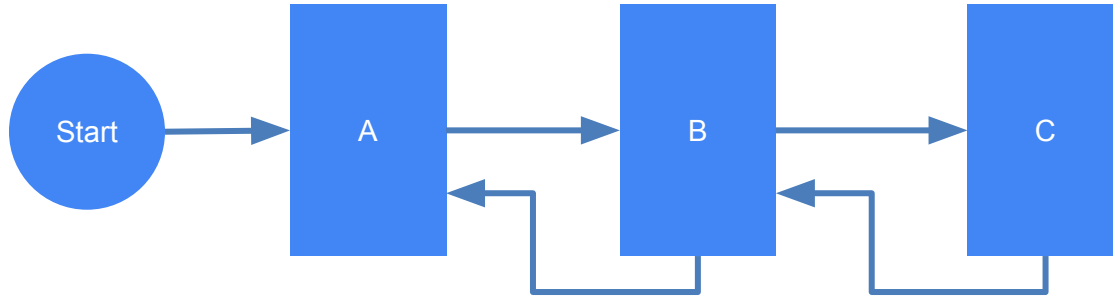
Navigation Principles

1. Fixed start destination
2. Navigation state is represented as a stack of destinations
3. Up and Back are identical within your app's task
4. The Up button never exits your app
5. Deep linking simulates manual navigation

Navigation Principles

1. Fixed start destination

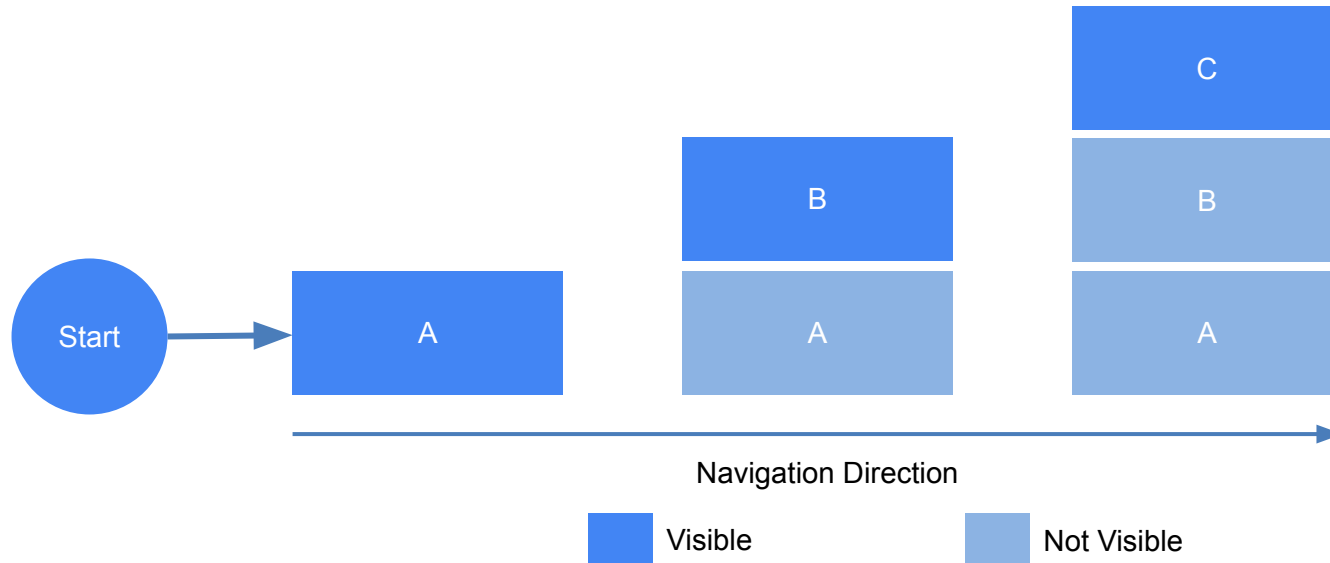
Every app you build has a fixed start destination. This destination is also the last screen the user sees when they return to the launcher after pressing the Back button.



Navigation Principles

2. Navigation state is represented as a stack of destinations

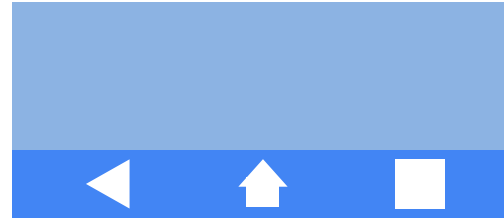
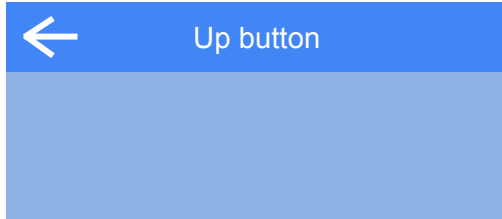
The top of the stack is the current screen, and the previous destinations in the stack represent the history of where you've been.



Navigation Principles

3. Up and Back are identical within your app's task

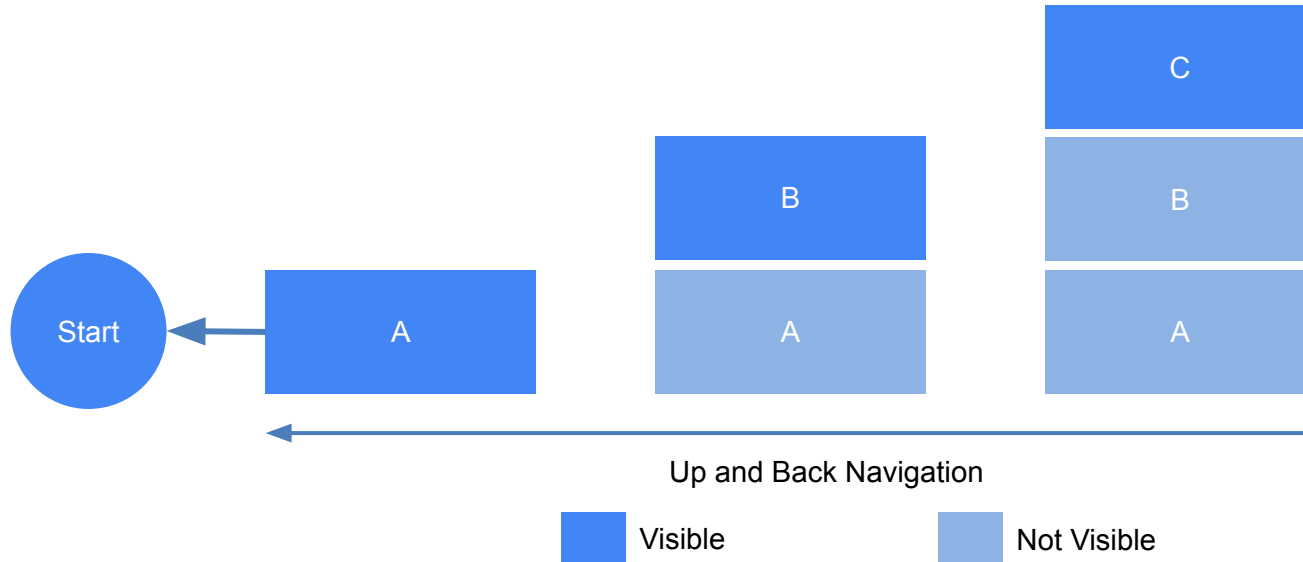
The Up button appears in the app bar at the top of the screen. Within your app's task, the Up and Back buttons behave identically.



Navigation Principles

3. Up and Back are identical within your app's task

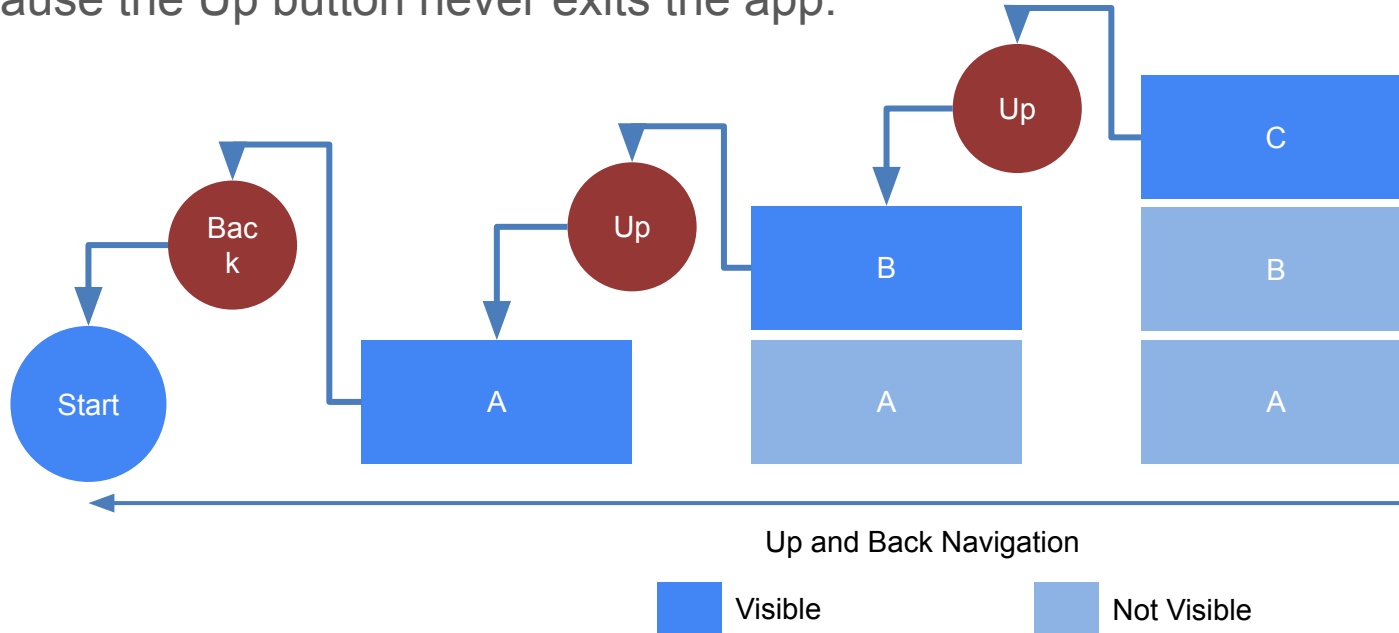
When you press the Back button, the current destination is popped off the top of the back stack, and you then navigate to the previous destination.



Navigation Principles

4. The Up button never exits your app

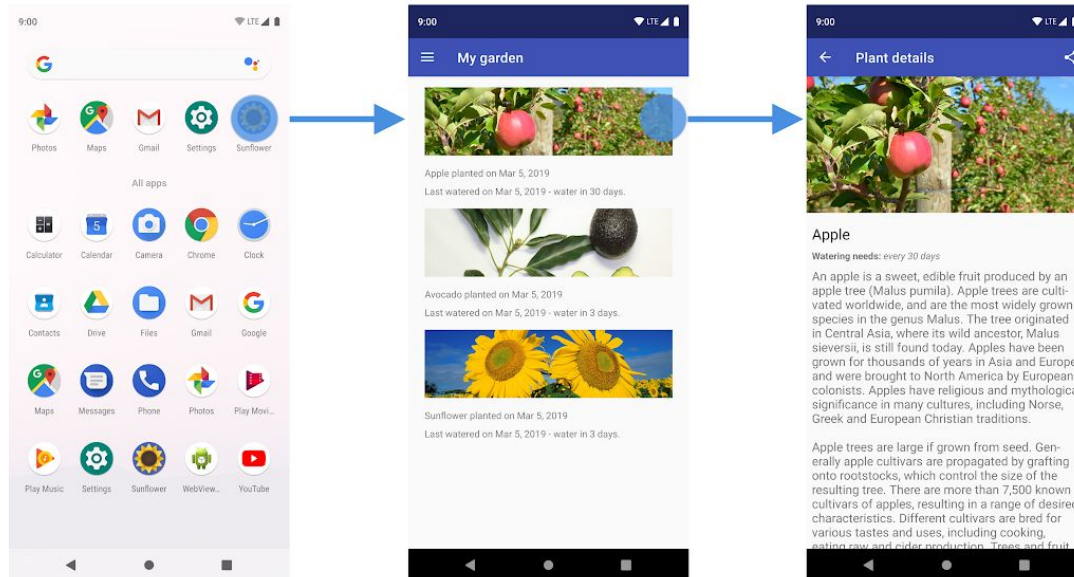
If a user is at the app's start destination, then the Up button does not appear, because the Up button never exits the app.



Navigation Principles

5. Deep linking simulates manual navigation

Navigation design should simulate manual navigation. E.g. From the app navigator to the main screen of an app, and then to a detail screen.



Tab

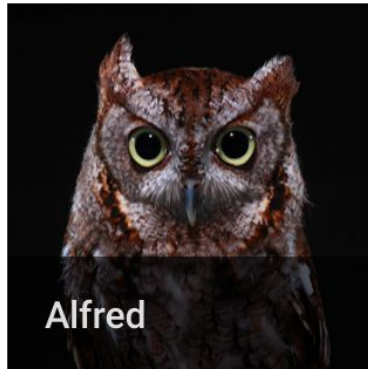
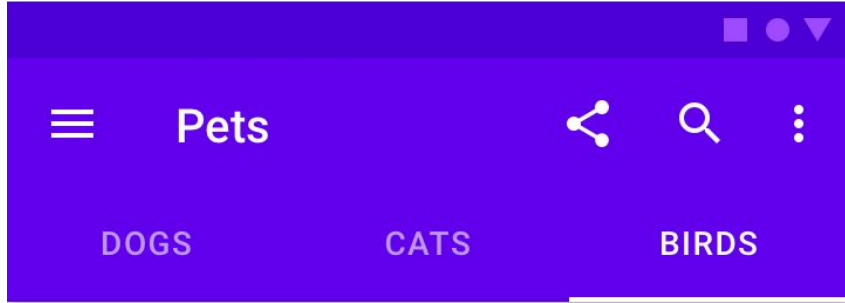
Tab

A `TabBar` and `TabBarView` combination is commonly used to create tabbed interfaces in Flutter.

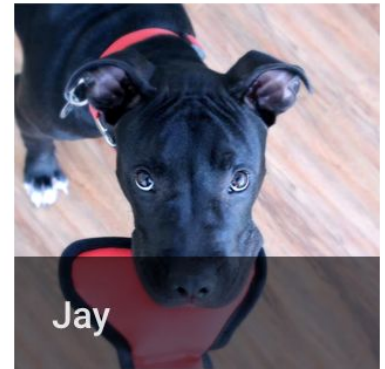
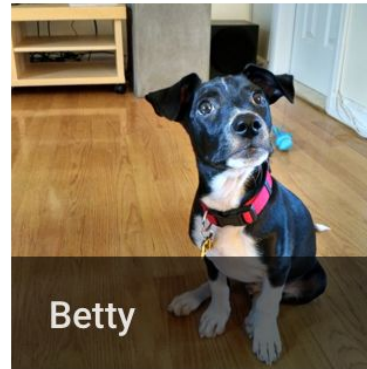
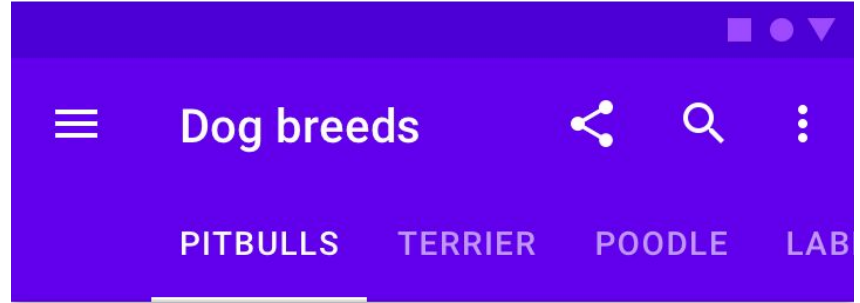
The `TabBar` displays the tabs, while the `TabBarView` displays the content associated with each tab.

The `TabController` manages the state of the tabs, allowing you to switch between them.

Tab



1. Fixed Tabs



2. Scrollable Tabs

Tab

Step 1: Create a TabController

```
return MaterialApp(  
  home: DefaultTabController(  
    length: 3,  
    child: Scaffold(),  
  ),  
);
```

Tab

Step 2: Create the tabs

```
return MaterialApp(  
  home: DefaultTabController(  
    length: 3,  
    child: Scaffold(  
      appBar: AppBar(  
        bottom: const TabBar(  
          tabs: [  
            Tab(icon: Icon(Icons.directions_car)),  
            Tab(icon: Icon(Icons.directions_transit)),  
            Tab(icon: Icon(Icons.directions_bike)),  
          ],  
        ),  
      ),  
    ),  
  ),  
);
```

Tab

Step 2: Create content for each tab

```
body: const TabBarView(  
  children: [  
    Icon(Icons.directions_car),  
    Icon(Icons.directions_transit),  
    Icon(Icons.directions_bike),  
  ],  
),
```

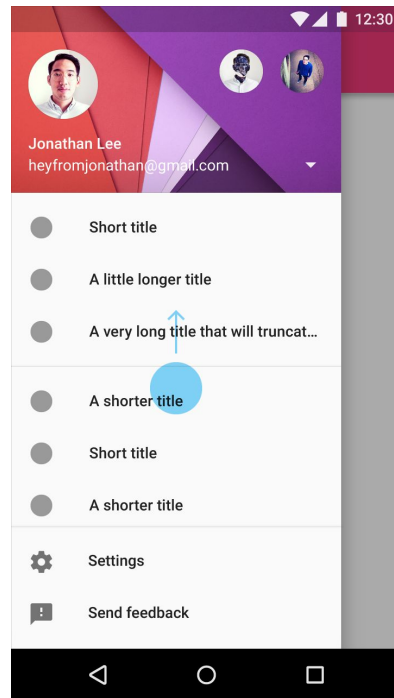
Drawer

Drawer

In Flutter, use the Drawer widget in combination with a Scaffold to create a layout with a Material Design drawer.

This recipe uses the following steps:

1. Create a Scaffold.
2. Add a drawer.
3. Populate the drawer with items.
4. Close the drawer programmatically.

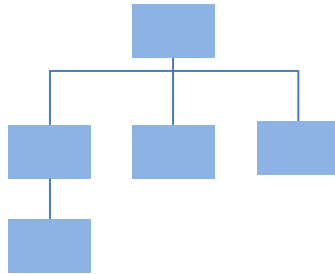


Navigation Drawer

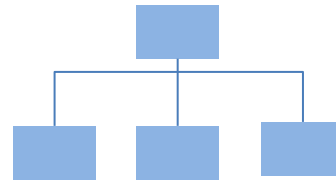
Recommended for:



≥ 5 top-level destinations



≥ 2 levels of navigation hierarchy



Quick navigation between unrelated destinations

Drawer

Step 1: Create a Scaffold

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('AppBar without hamburger button'),  
  ),  
  
);
```

Drawer

Step 2: Add a drawer

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('AppBar with hamburger button'),  
  ),  
  drawer: Drawer(  
    child: // Populate the Drawer in the next step.  
  ),  
);
```

Drawer

Step 3: Populate the drawer with items

```
Drawer(  
  // Add a ListView to the drawer.  
  child: ListView(  
    // Important: Remove any padding from the ListView.  
    padding: EdgeInsets.zero,  
    children: [  
      const DrawerHeader(  
        decoration: BoxDecoration(color: Colors.blue,),child: Text('Drawer Header'),),  
      ListTile(title: const Text('Item 1'), onTap: () { ... },),  
      ListTile(title: const Text('Item 2'), onTap: () { ... },),  
    ],  
  ),//End of ListView  
)//End of Drawer
```

Drawer

Step 4: Open the drawer programmatically.

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('AppBar with hamburger button'),  
    leading: Builder(  
      builder: (context) {  
        return IconButton(  
          icon: const Icon(Icons.menu),  
          onPressed: () { Scaffold.of(context).openDrawer(); },  
        );  
      },  
    ),  
  ),  
  drawer: Drawer(  
    child: // Populate the Drawer in the last step.  
  ),  
);
```

Drawer

Step 5: Close the drawer programmatically

```
ListTile(  
  title: const Text('Item 1'),  
  onTap: () {  
    // Update the state of the app  
    // ...  
    // Then close the drawer  
    Navigator.pop(context);  
  },  
)
```

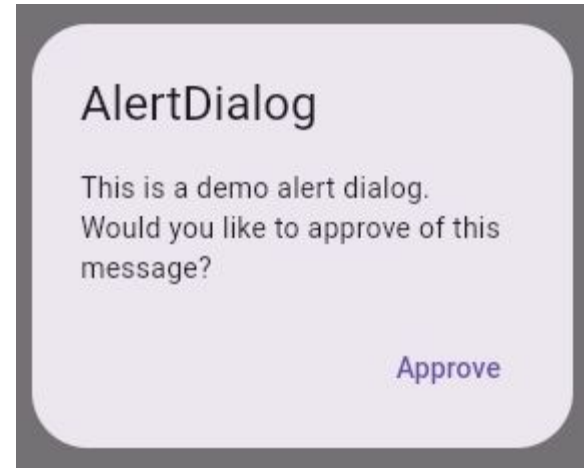
Dialog

AlertDialog

An alert dialog (also known as a basic dialog) informs the **user** about situations that require **acknowledgment**.

An alert dialog has an optional title and an optional list of actions.

The title is displayed above the content and the actions are displayed below the content.

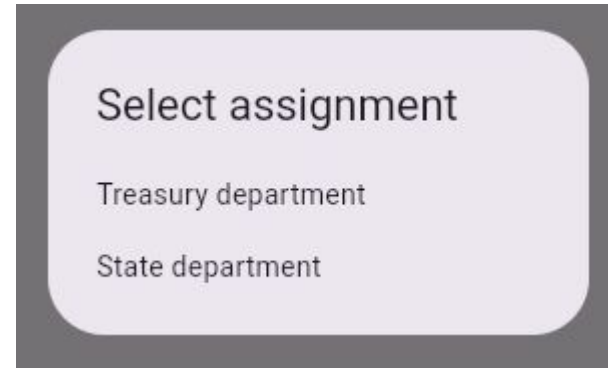


```
Future<void> _showMyDialog() async {  
  return showDialog<void>(  
    context: context, barrierDismissible: false, // user must tap button!  
    builder: (BuildContext context) {  
      return AlertDialog(  
        title: const Text('AlertDialog Title'),  
        content: const SingleChildScrollView(  
          child: ListBody(  
            children: <Widget>[Text('Would you like to approve of this message?'), ],  
          ),  
        ),  
        actions: <Widget>[  
          TextButton( child: const Text('Approve'),  
            onPressed: () {  
              Navigator.of(context).pop();  
            },  
          ),  
        ],  
      ),  
    ),  
  );  
}
```


SimpleDialog

A simple dialog offers the **user** a **choice** between several options.

A simple dialog has an optional title that is displayed above the choices.



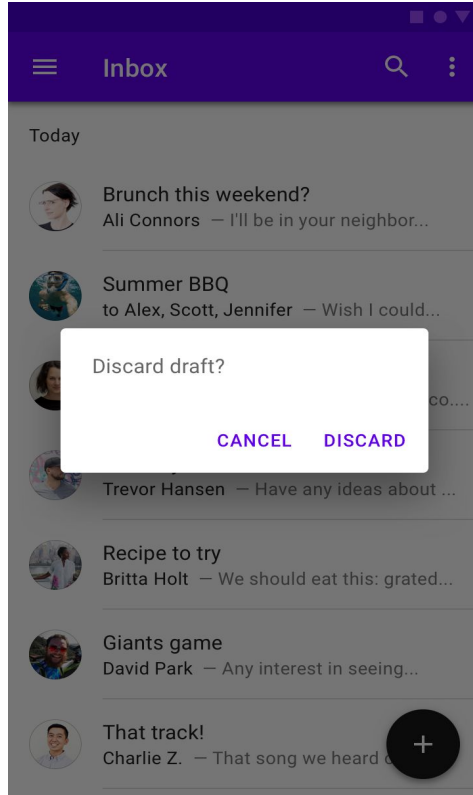
```

Future<void> _askedToLead() async {
  switch (await showDialog<Department>(
    context: context,
    builder: (BuildContext context) {
      return SimpleDialog(
        title: const Text('Select assignment'),
        children: <Widget>[
          SimpleDialogOption(
            onPressed: () { Navigator.pop(context, Department.treasury); },
            child: const Text('Treasury department'),
          ),
          SimpleDialogOption(
            onPressed: () { Navigator.pop(context, Department.state); },
            child: const Text('State department'),
          ),
        ],
      );
    }
  )) {
    case Department.treasury:
      // Let's go.
      // ...

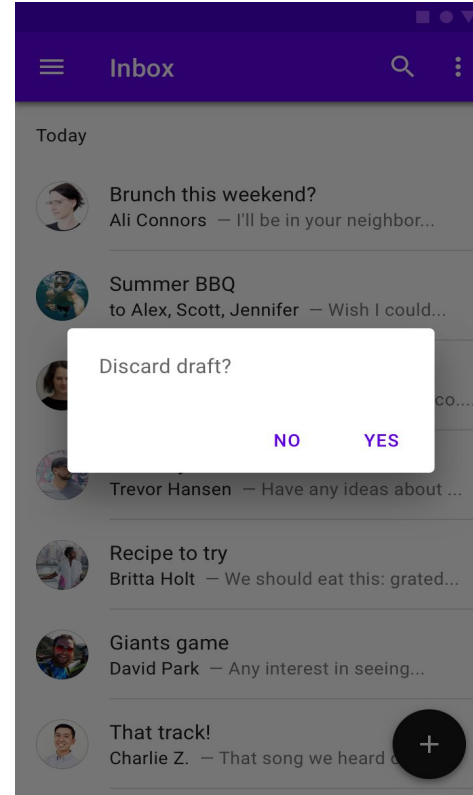
```

Which AlertDialog is better? Why?

(A)



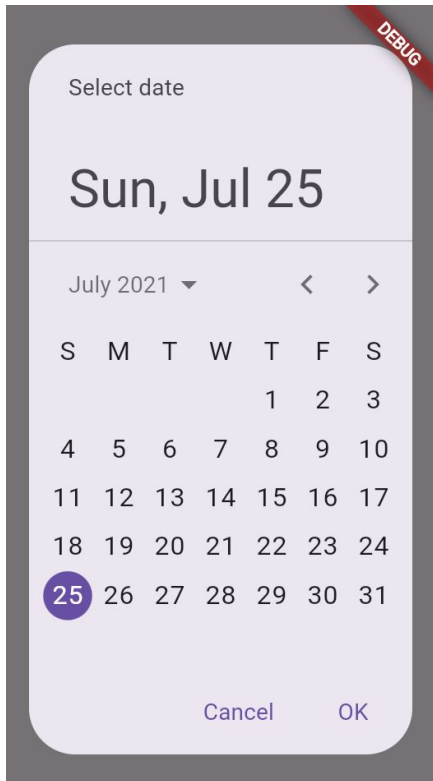
(B)



Date and Time Pickers

Using `showDatePicker` and `showTimePicker`.

Using these pickers helps ensure that your users can pick a time or date that is valid, formatted correctly, and adjusted to the user's locale.



Date and Time Pickers

```
DateTime _selectedDate = DateTime.now();
```

```
Future<void> _selectDate(BuildContext context) async {  
  final DateTime? picked = await showDatePicker(  
    context: context,  
    initialDate: _selectedDate,  
    firstDate: DateTime(2000),  
    lastDate: DateTime(2100),  
  );  
  if (picked != null) {  
    setState(() {  
      _selectedDate = picked;  
    });  
  }  
}
```



Date and Time Pickers

```
OutlinedButton(  
  onPressed: () {  
    _selectDate(context);  
  },  
  child: Text('$_selectedDate'),  
),
```

2024-12-03 11:21:39.006

Snackbar

Inform your users when certain actions take place.

For example, a message has been deleted.

Give the user an option to undo an action.

Snackbar

Insert a snackbar within the Scaffold.

```
final snackBar = SnackBar(  
  content: const Text('Yay! A SnackBar!'),  
  action: SnackBarAction(  
    label: 'Undo',  
    onPressed: () {  
      // Some code to undo the change.  
    },  
  ),  
);
```

```
// Find the ScaffoldMessenger in the widget tree and use it to show a SnackBar.  
ScaffoldMessenger.of(context).showSnackBar(snackBar);
```


Snackbar

Component	Priority	User Action
Snackbar	Low	Optional It disappears automatically
Dialog	High	Required It blocks app usage until the user takes a dialog action or exists the dialog

Find Out More

Tab, <https://docs.flutter.dev/cookbook/design/tabs>

Drawer, <https://docs.flutter.dev/cookbook/design/drawer>

AlertDialog, <https://api.flutter.dev/flutter/material/AlertDialog-class.html>

SimpleDialog, <https://api.flutter.dev/flutter/material/SimpleDialog-class.html>

Snackbar, <https://docs.flutter.dev/cookbook/design/snackbars>